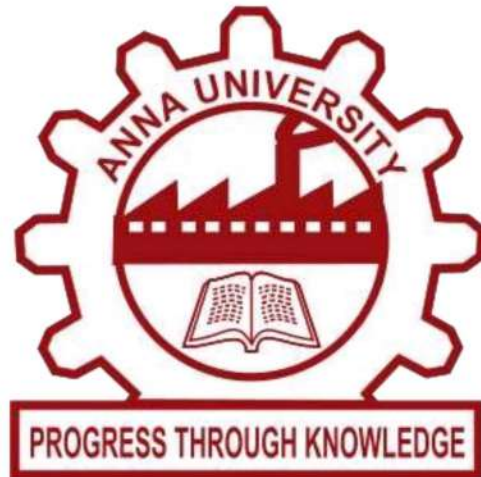


UNIVERSITY COLLEGE OF ENGINEERING NAGERCOIL

(ANNA UNIVERSITY CONSTITUENT COLLEGE)

KONAM, NAGERCOIL - 629004



RECORD NOTE BOOK

**EMBEDDED SYSTEMS
AND IOT - CS3691**

Register No : _____

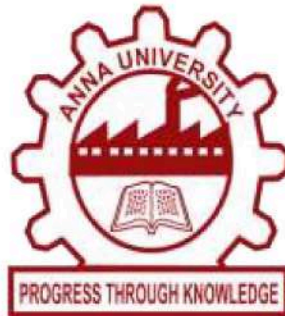
Name : _____

Year/Semester : _____

Department : _____

UNIVERSITY COLLEGE OF ENGINEERING NAGERCOIL

(ANNA UNIVERSITY CONSTITUENT COLLEGE)
KONAM, NAGERCOIL -629004



Register No:

*Certified that, this is the bonafide record of work done by
Mr. / Ms. Of VI
Semester in Computer Science and Engineering of this
college, in the Embedded Systems and Iot (CS3691) during
academic year (2025-2026) in partial fulfilment of the
requirements of the B.E Degree course of the Anna University
Chennai.*

Staff-in-charge

Head of the Department

This record is submitted for the University Practical Examination
held on

Internal Examiner

External Examiner

LIST OF EXPERIMENTS

[illegible]

[illegible]

EX NO :

DATE :

**WRITE 8051 ASSEMBLY LANGUAGE
EXPERIMENTS USING KEIL SIMULATOR**

Aim:

To write 8051 assembly language experiments using simulator.

Apparatus required:

Keil μ software

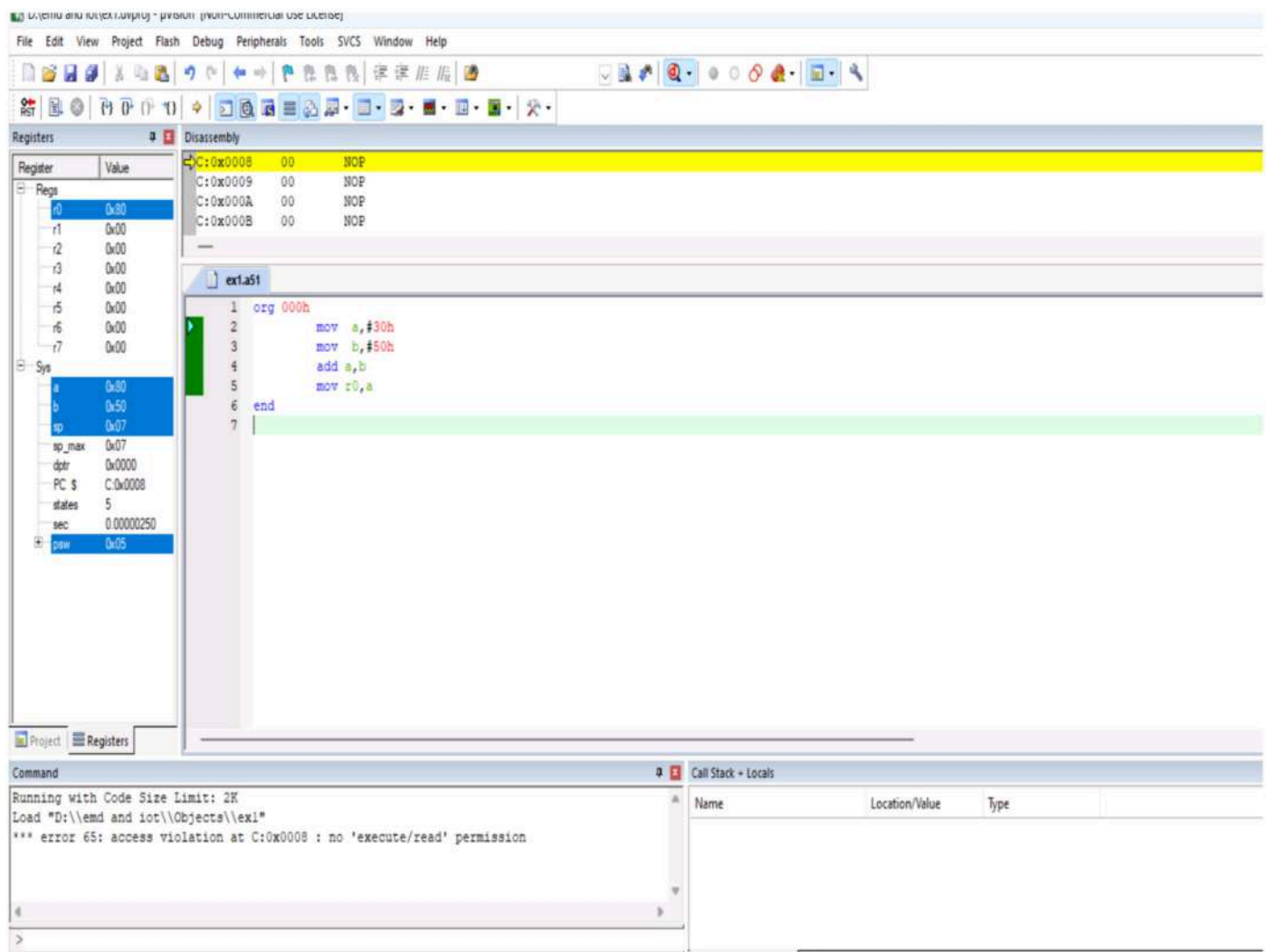
Procedure:

1. The 8051 microcontroller has a set of instructions for arithmetic, logic, and data manipulation operations. that demonstrates some basic ALU operations using assembly language for the 8051 microcontroller.
2. The various ALU operations, including addition, subtraction, multiplication, division, logical AND/OR/XOR, and rotation. Note that the specific instructions and registers used may vary depending on the exact 8051 microcontroller variant you are working with.
3. Always refer to the datasheet or reference manual for your specific microcontroller to ensure accurate programming.

Program:

```
org 000h  
  
mov a,#30h  
  
mov b,#50h  
  
add a,b  
  
mov r0,a  
  
end
```

Output:



The screenshot displays the Icy8051 IDE interface during a simulation. The top menu bar includes File, Edit, View, Project, Flash, Debug, Peripherals, Tools, SVCS, Window, and Help. The main workspace is divided into several panels:

- Registers:** A table showing the state of registers. General registers r0-r7 are all 0x00. System registers include a (0x50), b (0x50), sp (0x07), sp_max (0x07), dptr (0x0000), PC (C:0x0008), states (5), sec (0.00000250), and paw (0x05).
- Disassembly:** Shows four NOP instructions at memory locations C:0x0008 through C:0x000B.
- Source:** Displays the assembly code for 'ext1a51'. The code includes:

```
1 org 000h
2     mov a, #30h
3     mov b, #50h
4     add a, b
5     mov r0, a
6 end
```
- Command:** Shows the execution log with the message: "Running with Code Size Limit: 2K", "Load 'D:\\emd and iot\\Objects\\ext1'", and a critical error: "*** error 65: access violation at C:0x0008 : no 'execute/read' permission".
- Call Stack + Locals:** An empty table with columns for Name, Location/Value, and Type.

Result:

Thus the 8051 assembly language program was successfully implemented and simulated.

EX NO :

DATE :

WRITE 8051 ASSEMBLY LANGUAGE EXPIREMENTS USING EDSIM 51 SIMULATOR

Aim:

The purpose of this experiment is to write and implement 8051 assembly language experiments using simulator.

Apparatus required:

Edsim51 simulator

Program logic:

An Assembly language program consists of, among other things, a series of line of Assemble language instructions. An Assembly language instruction consists of mnemonic, optionally followed by one or two operands. The operands are the data items being manipulated and the mnemonics are the commands to the CPU, telling it what to do with those items.

☐ **Looping:**

Repeating a sequence of instructions a certain number of time is called a loop. The loop is one of the most widely used actions that any microprocessor performs. In this instruction, the register is decremented; if it is not zero, it jumps to the target address referred by the label. Prior to the start of the loop the register is loaded with the counter for the number of repetitions. In this instruction both the register decrement and the decision to jump are combined into a single instruction.

A loop inside another loop is called nested loop.

☐ **Unconditional jump instruction:**

The unconditional jump is a jump in which control is transferred unconditionally to the target location. In 8051 there is two unconditional jumps:

- **LJMP(long jump):**

LJMP is an unconditional long jump. It is a 3-byte instruction in which the first byte is a opcode, second and third bytes represent the 16-bit address of the target location. The 2-byte target address allows a

jump to any memory location from 0000 to FFFFH.

- **SJMP(Short jump):**

SJMP is an unconditional short jump. It is a 2-byte instruction in which the first byte is a opcode and second byte is the relative address of the target location. The target address ranges from 00 to FFH.

Procedure:

Step-1: Open the Edsim51 simulator.

Step-2: Type the program and save it with “.asm” or “.hex” extension.

Step-3: Run the program and rectify the result.

Step-4: Observe the output.

Step-5: Repeat the above steps for all the program.

Program:

1. Sample of Assembly language program:

```
ORG 0H
MOV R5, #25H
MOV R7, #34H
MOV A, #0
ADD A, R5
ADD A, R7
ADD A, #12H
HERE: SJMP HERE
END
```

2. Multiply 25 by 10 using the technique of repeated addition:

```
MOV A, #0
MOV R2, #10
AGAIN: ADD A, #25
DJNZ R2, AGAIN
MOV R5, A
```


3. Program to add first 10 natural number:

```
MOV A,#0
MOV R2, #10
MOV R0, #0
AGAIN:INC R0
ADD A,R0
DJNZ R2, AGAIN
MOV 46H, A
```

4. Program to load the accumulator with the value of 55H and complement the ACC 700 times:

```
MOV A,#55H
MOV R3,#10
NEXT:MOV R2,#70
AGAIN:CPL A
DJNZ R2,AGAIN
DJNZ R3,NEXT
```

5. Multiply ECH by 25H using the technique of repeated addition:

```
MOV R1,#0
MOV A,#0
MOV R0,#25H
AGAIN: ADD A,#0ECH
JNC HERE
INC R1
HERE: DJNZ R0,AGAIN
MOV R0,A
```

Output:

1. Sample of Assembly language program:

The screenshot displays the Proteus IDE interface for an 8051 microcontroller simulation. The top-left pane shows the register file with the PC (Program Counter) at 0x000A and the SP (Stack Pointer) at 0x007. The top-right pane shows the assembly code:

```
ORG 0H
0000: MOV R5, #25H
0002: MOV R7, #34H
0004: MOV A, #0
0006: ADD A, R5
0007: ADD A, R7
0008: ADD A, #12H
000A: HERE: SJMP HERE
END
```

The bottom pane shows the hardware interface, including a keypad, a 7-segment display, and various control buttons. The 7-segment display shows the value 8888.

2. Multiply 25 by 10 using the technique of repeated addition:

The screenshot displays the Proteus IDE interface for an 8051 microcontroller simulation. The top-left pane shows the register file with the PC (Program Counter) at 0x002C and the SP (Stack Pointer) at 0x007. The top-right pane shows the assembly code:

```
0000: MOV A, #0
0002: MOV R2, #10
0004: AGAIN: ADD A, #25
0006: DJNZ R2, AGAIN
0008: MOV R5, A
```

The bottom pane shows the hardware interface, including a keypad, a 7-segment display, and various control buttons. The 7-segment display shows the value 8888.

3. Program to add first 10 natural number:

The screenshot displays the Proteus 8.10 SP3 environment. On the left, the Register File shows R0=0x0A, R1=0x00, R2=0x00, R3=0x00, R4=0x00, R5=0xFA, R6=0x00, R7=0x34. The Accumulator (ACC) is 0x37. The Program Counter (PC) is 0x005B. The Data Memory shows the sum 55H at address 0x00. The assembly code in the center is as follows:

```

0000| MOV A, 0
0002| MOV R2, #10
0004| MOV R0, #0
0006| AGAIN: INC R0
0007| ADD A, R0
0008| DJNZ R2, AGAIN
000A| MOV 46H, A
  
```

The hardware interface at the bottom shows a keypad with digits 0-9, a display showing 55, and various control buttons like 'AND Gate Disabled', 'Key Bounce Disabled', 'Standard', 'Rx Reset', 'Tx Send', '0.0V output', 'Scope', 'MAX', and 'MIN'.

4. Program to load the accumulator with the value of 55H and complement the ACC 700 times:

The screenshot displays the Proteus 8.10 SP3 environment. On the left, the Register File shows R0=0x0A, R1=0x00, R2=0x00, R3=0x00, R4=0x00, R5=0xFA, R6=0x00, R7=0x34. The Accumulator (ACC) is 0x55. The Program Counter (PC) is 0x0006. The Data Memory shows the value 55H at address 0x00. The assembly code in the center is as follows:

```

0000| MOV A, #55H
0002| MOV R3, #10
0004| NEXT: MOV R2, #700
0006| AGAIN: CPL A
0007| DJNZ R2, AGAIN
0009| DJNZ R3, NEXT
  
```

The hardware interface at the bottom shows a keypad with digits 0-9, a display showing 55, and various control buttons like 'AND Gate Disabled', 'Key Bounce Disabled', 'Standard', 'Rx Reset', 'Tx Send', '0.0V output', 'Scope', 'MAX', and 'MIN'.

5. Multiply ECH by 25H using the technique of repeated addition:

The screenshot displays the 8051 simulator interface. The top section shows the register file with R0 at 0x1C and R1 at 0x22. The PC register is at 0x0022. The instruction register shows the current instruction: `MOV R0, A`. The assembly code window contains the following program:

```
0000| MOV R1, #0
0002| MOV A, #0
0004| MOV R0, #25H
0006| AGAIN: ADD A, #0ECH
0008| JNC HERE
000A| INC R1
000B| HERE: DJNZ R0, AGAIN
000D| MOV R0, A
```

The bottom section shows the hardware status. The DAC output is 0.0V. The ADC input is 11111111. The motor is enabled. The UART is configured for 8-bit, No Parity, 4800 Baud. The keypad is disabled.

Result:

Thus the 8051 assembly level program was written and implemented successfully

EX NO :

DATE :

TEST DATA TRANSFER BETWEEN REGISTER AND MEMORY USING KEIL SIMULATOR

Aim:

To test data transfer between register and memory using keil simulator

Apparatus required:

Keil μ software

Procedure:

1. The 8051 microcontroller has a set of instructions for arithmetic, logic, and data manipulation operations. that demonstrates some basic ALU operations using assembly language for the 8051 microcontroller.
2. The various ALU operations, including addition, subtraction, multiplication, division, logical AND/OR/XOR, and rotation. Note that the specific instructions and registers used may vary depending on the exact 8051 microcontroller variant you are working with.
3. Always refer to the datasheet or reference manual for your specific microcontroller to ensure accurate programming.

Program:

```
ORG 0000H

MOV R0,#30H

MOV R1,#50H

MOV R3,#05H

BACK: MOV A,@R0

MOV @R1,A

INC R0

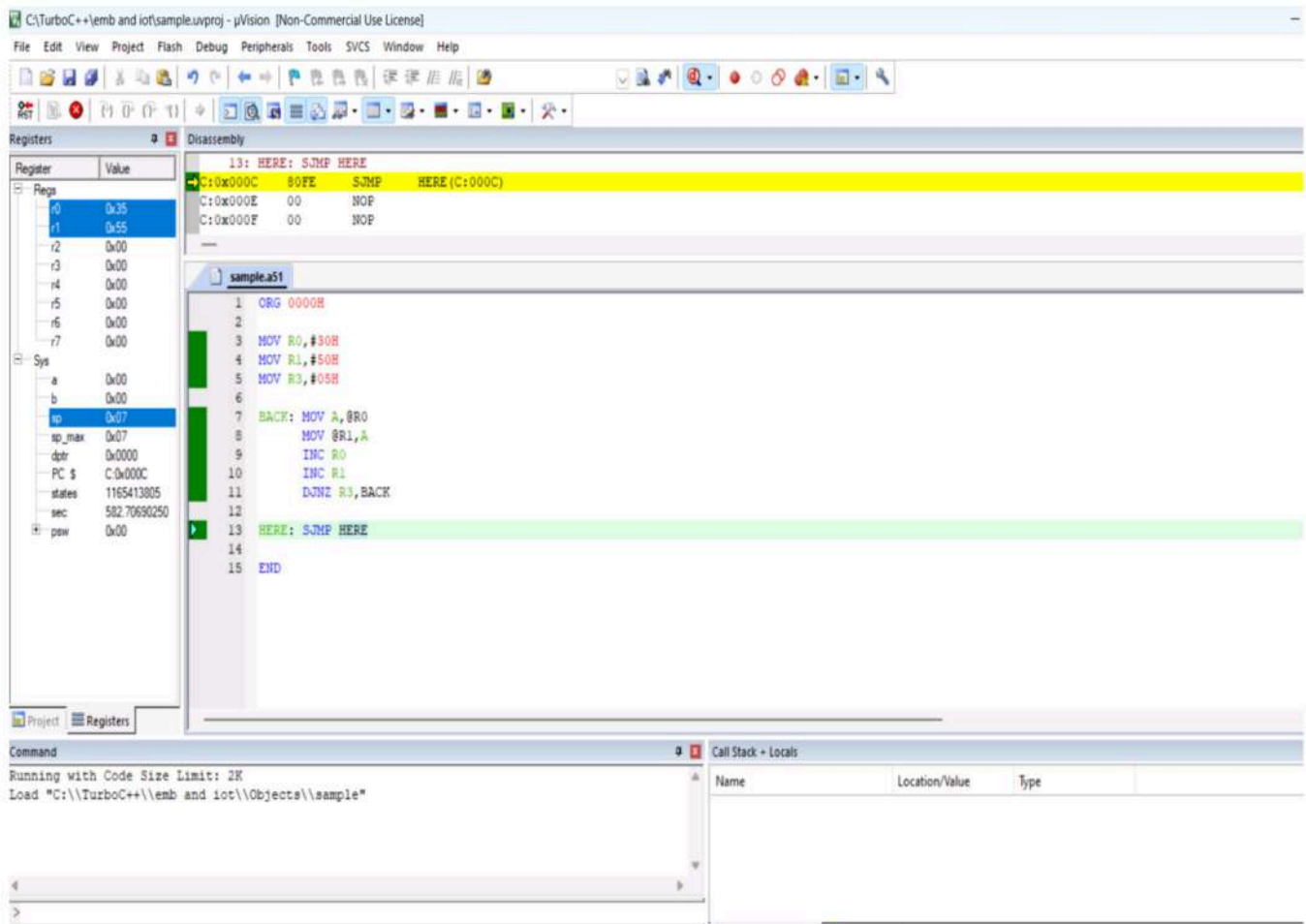
INC R1

DJNZ R3,BACK

HERE: SJMP HERE
```

END

Output:



Result:

The program successfully performed simple data transfer from one memory location to another using registers.

EX NO :

DATE :

TO TEST DATA TRANSFER BETWEEN REGISTER AND MEMORY USING EDSIM 51 SIMULATOR

Aim:

The purpose of this experiment is to test data transfer between register and memory.

Software required:

Edsim51 simulator.

Program logic:

In 8051, the action of comparing and jumping are combined into single instruction called CJNE. The CJNE instruction compares two operands and jump if they are not equal. In addition, it changes the CY flag to indicate if the destination operand is larger or smaller. It is important to notice that the operand themselves remains unchanged.

Procedure:

Step-1: Enter the opcode in Edsim51 simulator.

Step-2: Execute the program.

Step-3: Check the result in register A.

Program:

MOV R0,#50H

MOV R1,#10H

MOV B,#0

BACK: MOV A,@R0

CJNE A,B,LOOP

LOOP: JC LOOP1

MOV B,A

INC R0

DJNZ R1,BACK

SJMP NEXT

LOOP1: INC R0

DJNZ R1,BACK

NEXT: MOV A,B

MOV 60H,A

END

Output:

The screenshot displays the Proteus 8051 simulator interface. The central window shows the assembly code being executed, with the following instructions visible:

```
0000| MOV R0,#50H
0002| MOV R1,#10H
0004| MOV B,#0
0007| BACK: MOV A,@R0
0008| CJNE A,B,LOOP
000B| LOOP: JC LOOP1
000D| MOV B,A
000F| INC R0
0010| DJNZ R1,BACK
0012| SJMP NEXT
0014| LOOP1: INC R0
0015| DJNZ R1,BACK
0017| NEXT: MOV A,B
0019| MOV 60H,A
END
```

The left panel shows the 8051 register file and memory. The PC register is at 0x054B, and the PSW register is at 0x0000. The right panel shows the hardware components, including the Display-select Decoder CS|DAC WR, Keypad Column 2, Keypad Column 1, Keypad Column 0, Keypad Row 3, Keypad Row 2, Keypad Row 1, Keypad Row 0, LED 7|Seg. dp|DAC DB7|LCD DB7, LED 6|Seg. g|DAC DB6|LCD DB6, LED 5|Seg. f|DAC DB5|LCD DB5, LED 4|Seg. e|DAC DB4|LCD DB4, LED 3|... d|..DB3|..DB3|.. RS, LED 2|... c|..DB2|..DB2|LCD E, LED 1|Seg. b|DAC DB1|LCD DB1, LED 0|Seg. a|DAC DB0|LCD DB0, SW 7|ADC DB7, SW 6|ADC DB6, SW 5|ADC DB5, SW 4|ADC DB4, SW 3|ADC DB3, SW 2|ADC DB2, SW 1|ADC DB1, SW 0|ADC DB0, ADC RD|Comparator Output, ADC WR, Motor Sensor, Display-select Input 1, AND Gate Output|Display-se..t 0, ADC INTR, Motor Control Bit 1|Ext. UART Rx, and Motor Control Bit 0|Ext. UART Tx.

The bottom panel shows the hardware components, including the DI, LD, AND Gate Disabled, Key Bounce Disabled, Standard, Rx, Tx, Rx Reset, Tx Send, 0.0V input, 11111111, ADC, MAX, MIN, and Motor Enabled.

Result:

Thus the program to text data transfer between Register and Memory was written and executed successfully.

EX NO:

DATE:

PERFORM ALU OPERATIONS

Aim:

To perform ALU operations using Keil software.

Apparatus required:

Keil μ software

Procedure:

1. The 8051 microcontroller has a set of instructions for arithmetic, logic, and data manipulation operations that demonstrates some basic ALU operations using assembly language for the 8051 microcontroller.
2. The various ALU operations, including addition, subtraction, multiplication, division, logical AND/OR/XOR, and rotation. Note that the specific instructions and registers used may vary depending on the exact 8051 microcontroller variant you are working with.
3. Always refer to the datasheet or reference manual for your specific microcontroller to ensure accurate programming.

Program:

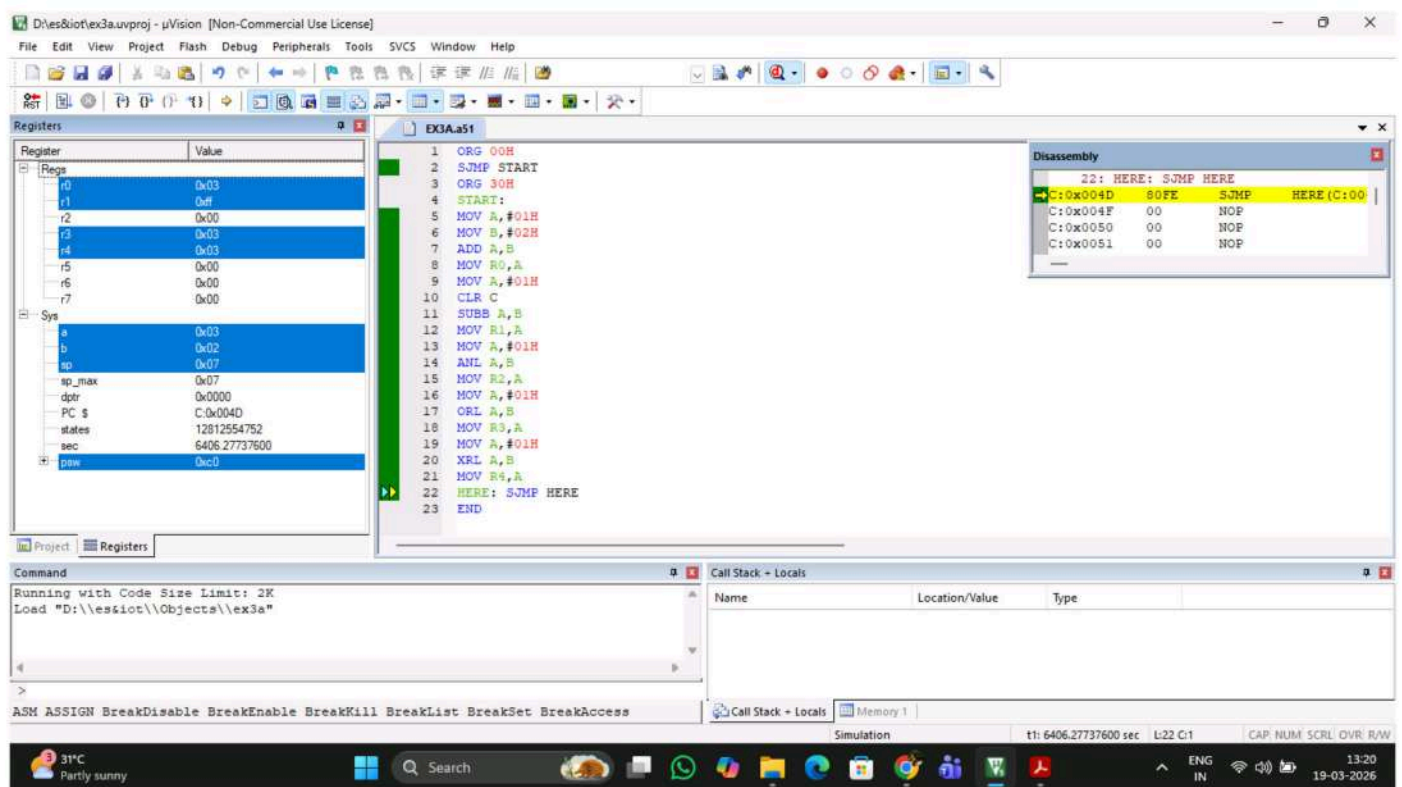
```
ORG 00H
SJMP START
ORG 30H
START:
MOV A,#01H
MOV B,#02H
ADD A,B
MOV R0,A
MOV A,#01H
CLR C
```

```

SUBB A,B
MOV R1,A
MOV A,#01H
ANL A,B
MOV R2,A
MOV A,#01H
ORL A,B
MOV R3,A
MOV A,#01H
XRL A,B
MOV R4,A
HERE: SJMP HERE
END

```

Output:



Result:

Thus the ALU operations using Keil software was performed and the result was successfully simulated.

EX NO:

DATE :

PERFORM ALU OPERATION USING 8051 MICROCONTROLLER

Aim:

The purpose of this experiment is to Add, Subtract, Multiply and Divide the given two 8 bit numbers and store them in a memory location using Edsim 51 Simulator.

Software requirement:

Edsim 51 Simulator

Procedure:

Step-1: Enter the opcodes from memory location 4200.

Step-2: Executes the program.

Step-3: Check for the result at 4100 and 4101. Using the accumulator, Subtraction is performed and the result is stored. Immediate addressing is employed. The SUBB instruction drives the result in the accumulator.

Program:

ADDITION:

```
CLR C           ; CY = 0
MOV A, #45H     ; Low byte 1
ADD A, #0ECH    ; Add low byte 2
MOV R0, A       ; Store low byte result
MOV A, #02H     ; High byte 1
ADDC A, #0FCH   ; Add high byte 2 WITH carry
MOV R1, A       ; Store high byte result
```

SUBTRACTION:

Subtraction with CY=0:

```
ORG 00H
MOV A, #50H      ; Load first number (minuend)
CLR C            ; Ensure CY = 0 (important!)
SUBB A, #20H     ; A = 50H - 20H
MOV R0, A        ; Store result
END
```

Subtraction with CY=1:

```
ORG 00H
MOV A, #50H      ; A = 50H
SETB C           ; CY = 1 (important!)
SUBB A, #20H     ; A = 50H - 20H - 1
MOV R0, A        ; store result
END
```

MULTIPLICATION:

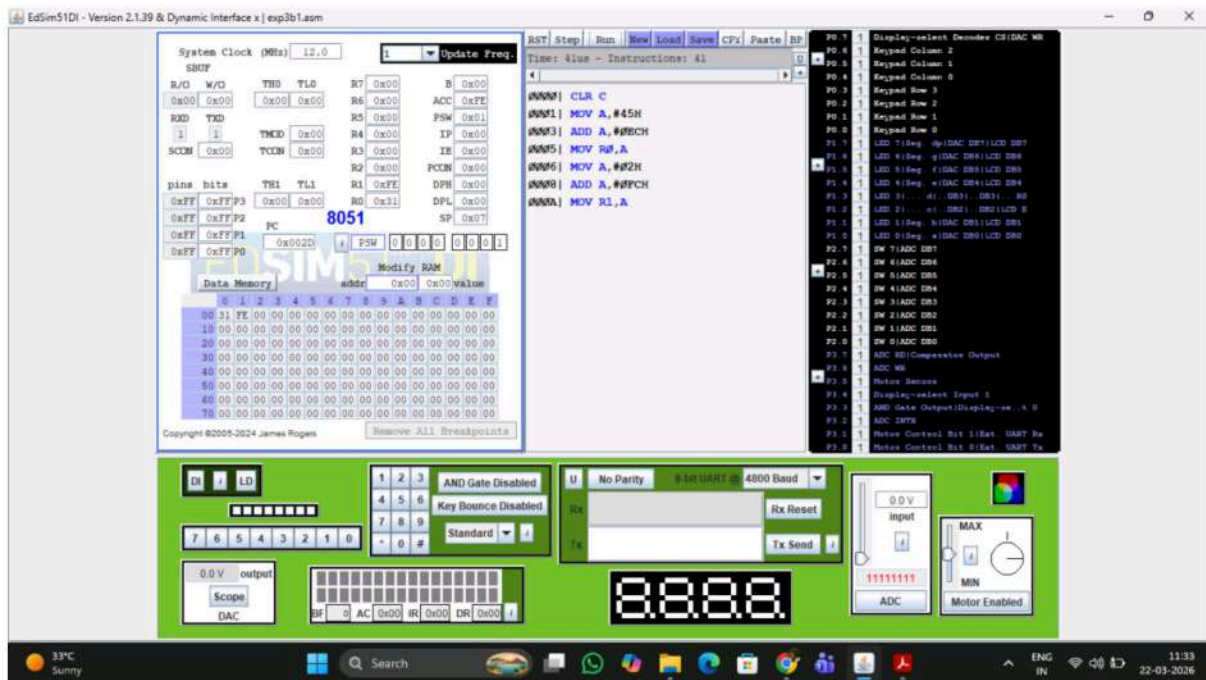
```
ORG 00H
MOV A, #05H      ; A = 5
MOV B, #04H      ; B = 4
MUL AB           ; A × B
MOV R0, A        ; store lower byte
MOV R1, B        ; store higher byte
END
```

DIVISION:

```
ORG 00H
MOV A, #14H      ; A = 20 (decimal)
MOV B, #05H      ; B = 5
DIV AB           ; A / B
MOV R0, A        ; Quotient
MOV R1, B        ; Remainder
END
```

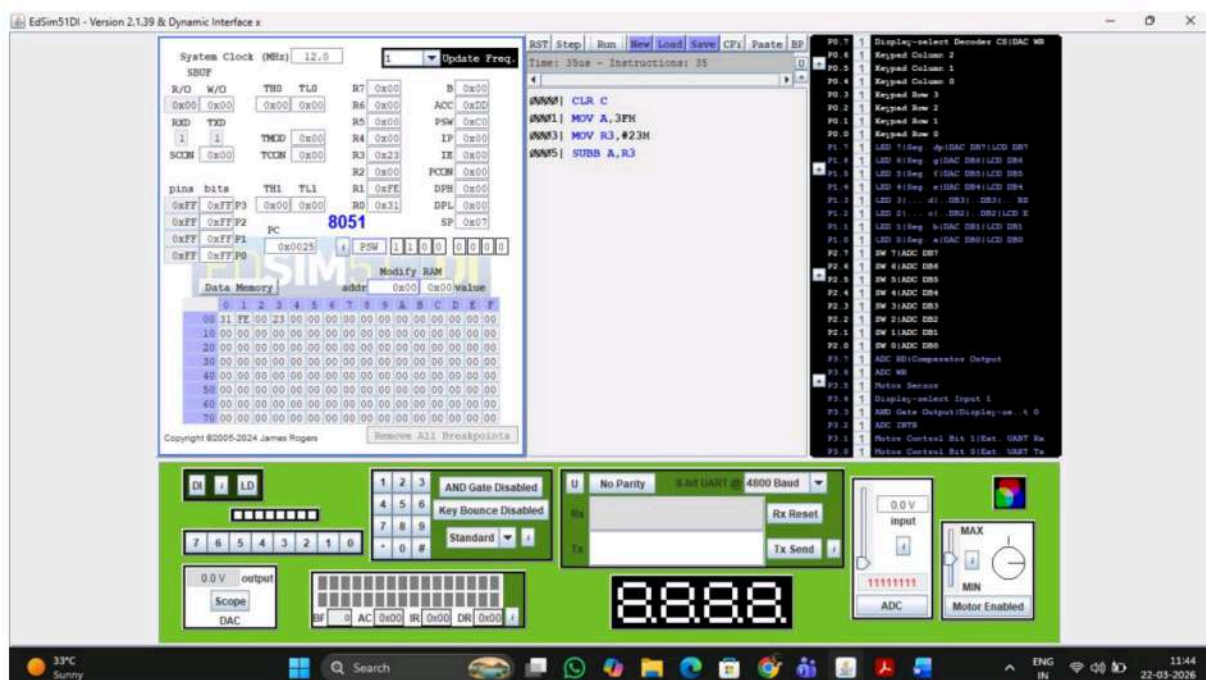
Output:

ADDITION:

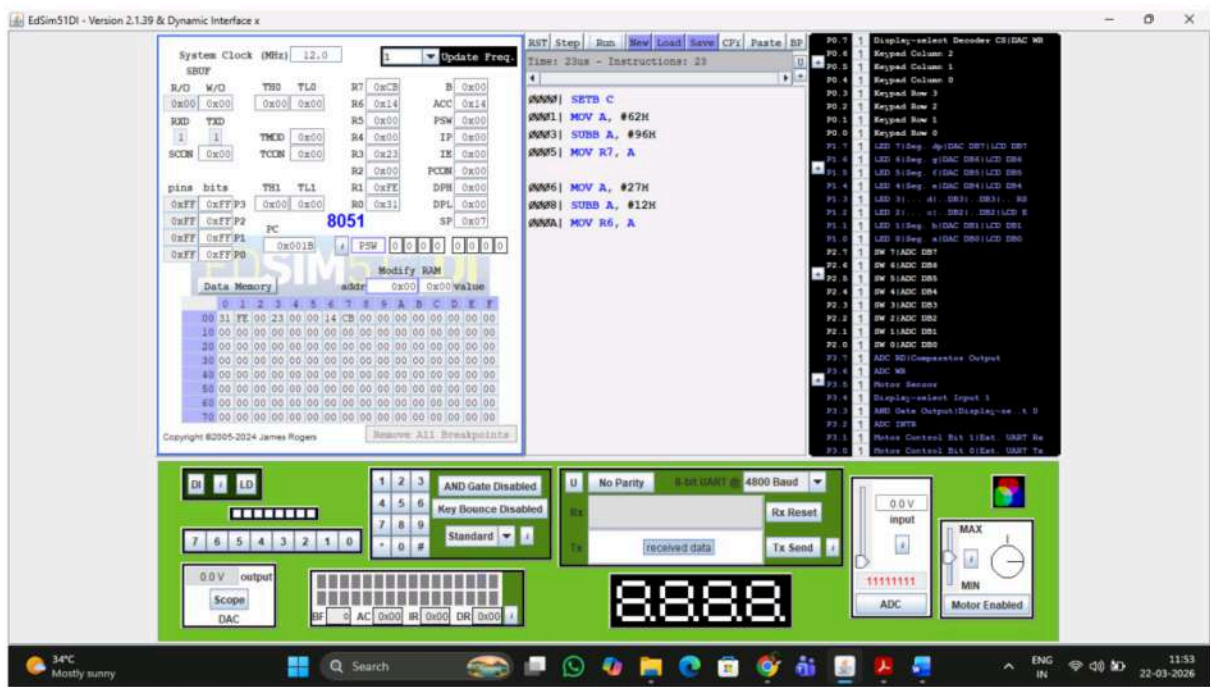


SUBTRACTION:

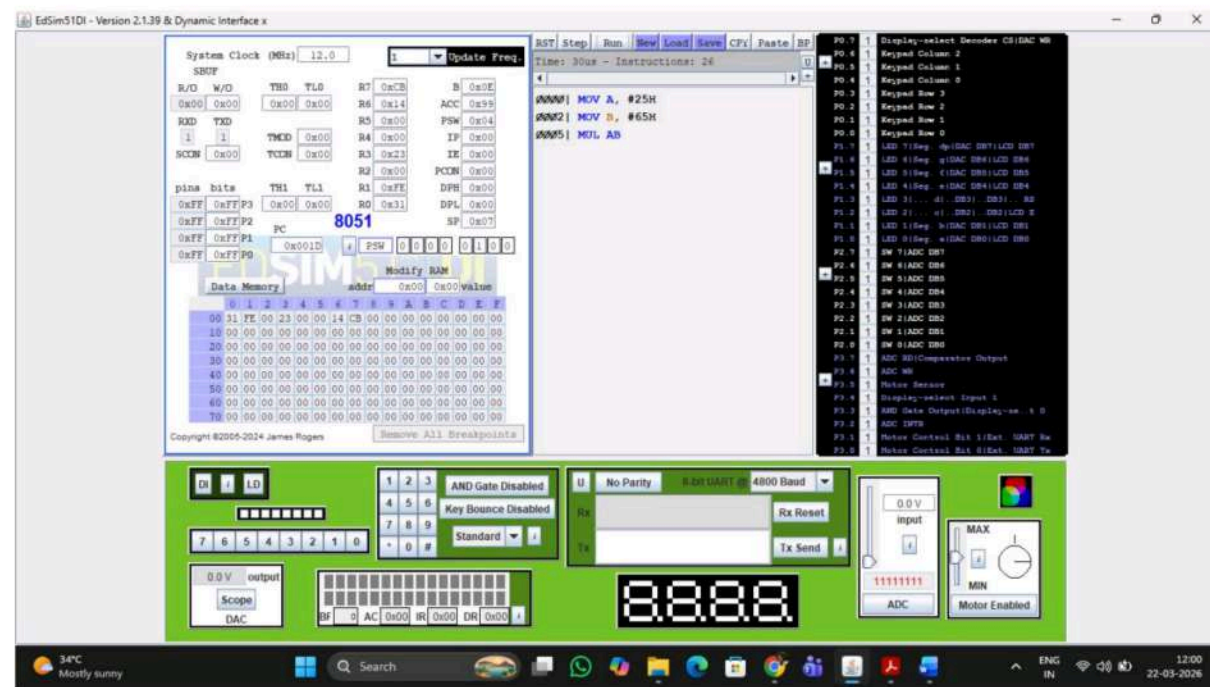
Subtraction with CY=0:



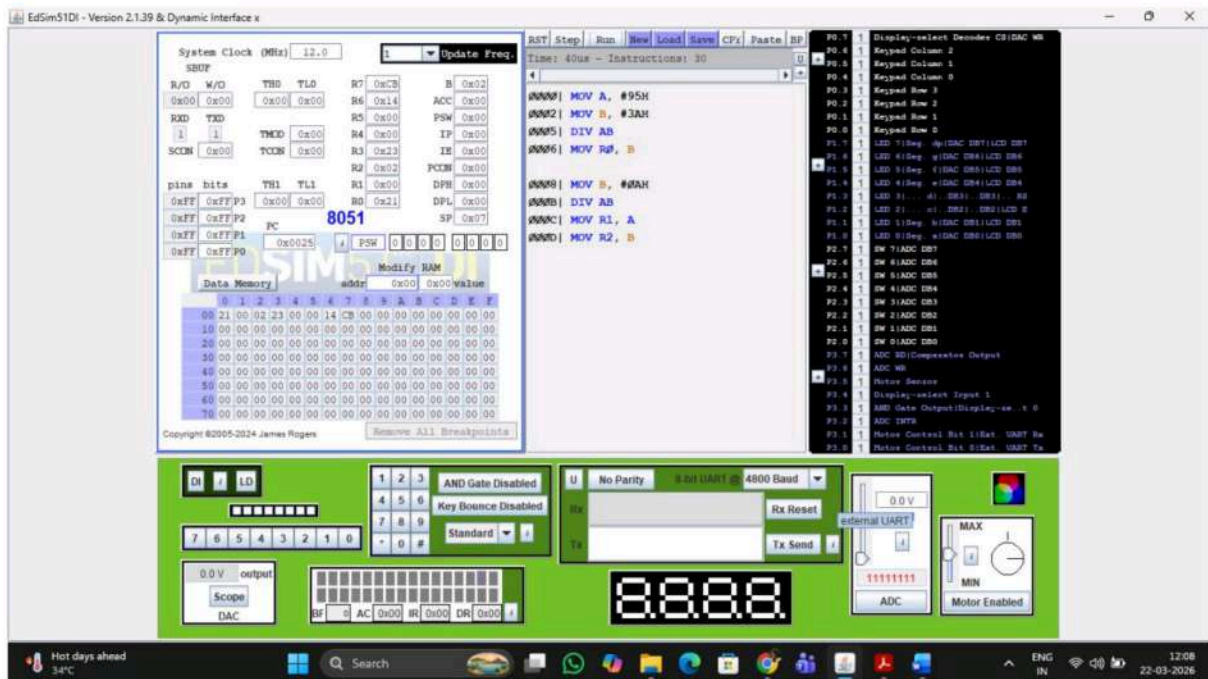
Subtraction with CY=1:



MULTIPLICATION:



DIVISION:



Result:

Thus the ALU operation using 8051 Microcontroller was performed and the result was obtained successfully.

EX NO:

DATE:

WRITE BASIC AND ARITHMETIC PROGRAM USING EMBEDDED C

Aim:

To write basic and arithmetic program Using Embedded C.

Apparatus required:

Keil μ software

Procedure:

- The 8051 microcontroller has a set of instructions for arithmetic, logic, and data manipulation operations. that demonstrates some basic ALU operations using assembly language for the 8051 microcontroller.
- The various ALU operations, including addition, subtraction, multiplication, division, logical AND/OR/XOR, and rotation. Note that the specific instructions and registers used may vary depending on the exact 8051 microcontroller variant you are working with.
- Always refer to the datasheet or reference manual for your specific microcontroller to ensure accurate programming.

Program:

```
#include<8051.h>

void main() {

    unsigned char a=0x0A;

    unsigned char b= 0x05;

    unsigned char result_addition, result_subtraction, result_multiplication, result_division;

    result_addition = a + b;
```

```

result_subtraction = a - b;

result_multiplication = a * b;

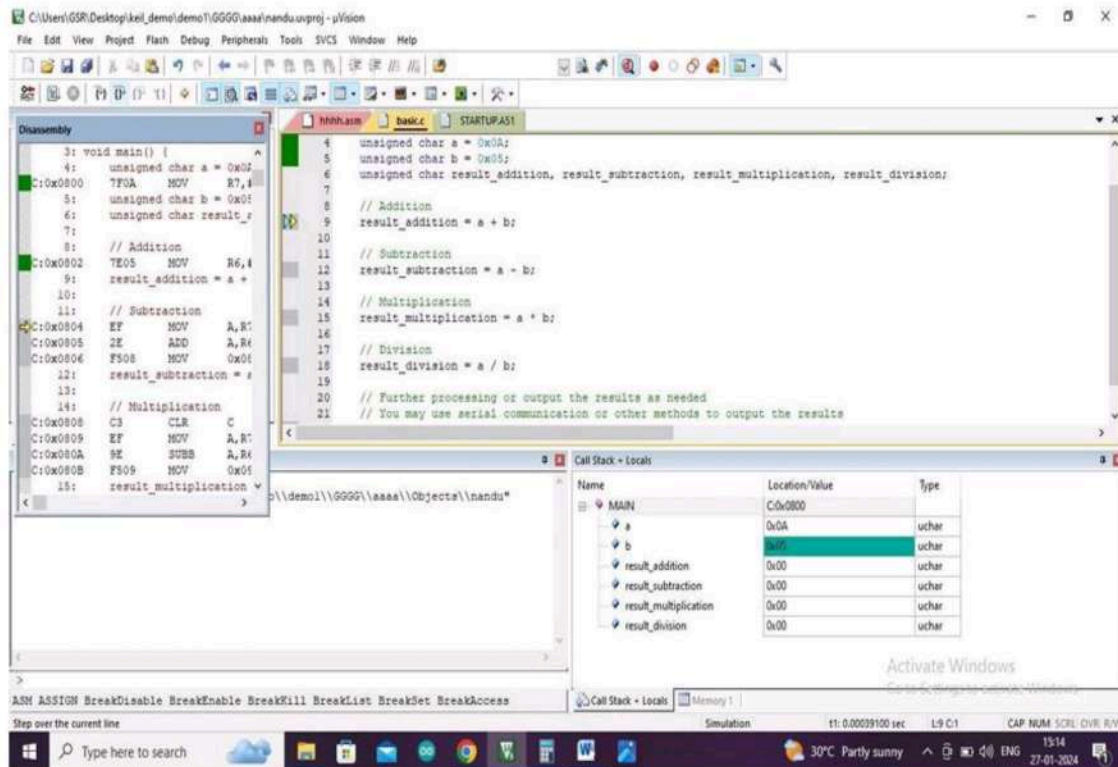
result_division = a / b;

}

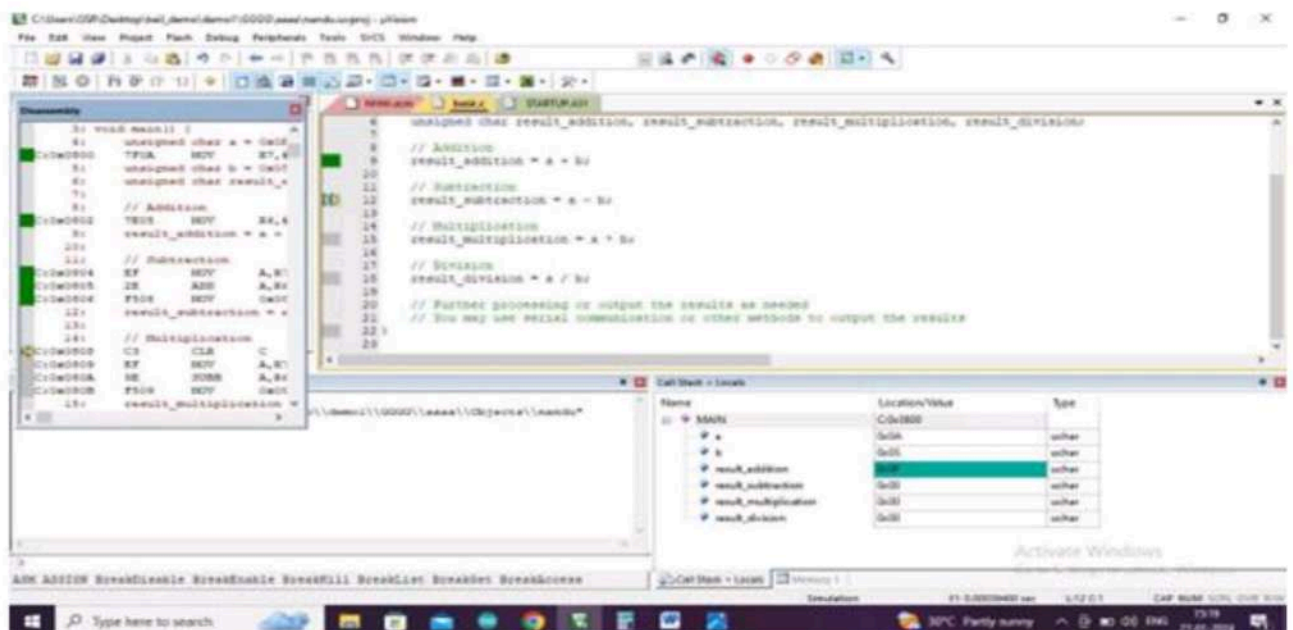
```

Output:

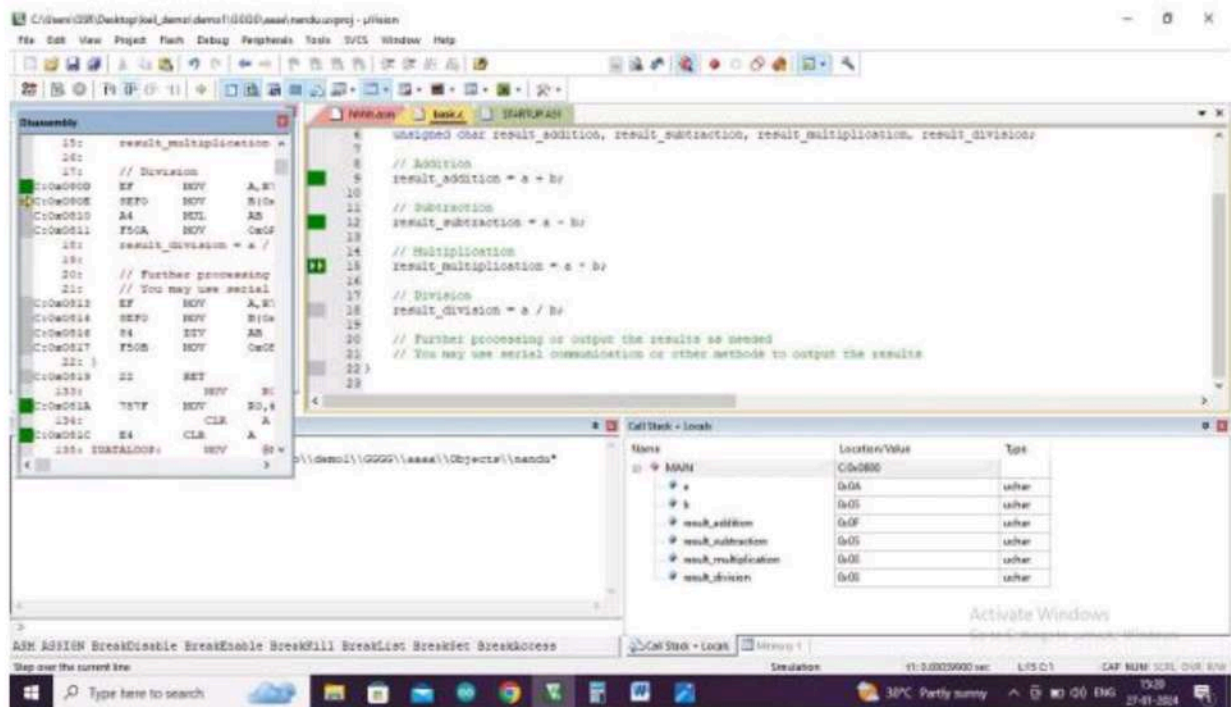
ADDITION:



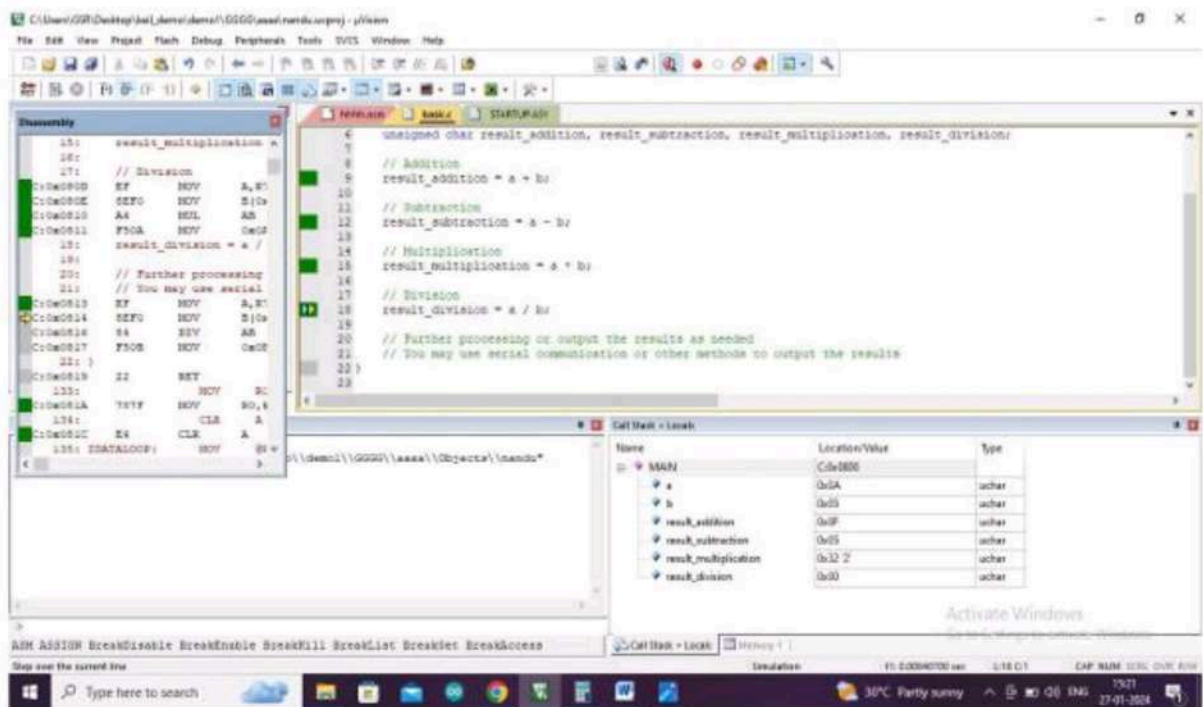
SUBTRACTION:



MULTIPLICATION:



DIVISION:



Another way

Program:

1.ADDITION:

```
#include <reg51.h>

void main(void)

{

    unsigned char x, y, z;

    x = 0x12;

    y = 0x34;

    P0 = 0x00;

    z = x + y;

    P0 = z;

}
```

2.SUBTRACTION:

```
#include <reg51.h>

void main(void)

{

    unsigned char a,b,c;

    a=0x12;

    b=0x34;

    P1=0x00;

    c=b-a;

    P1=c;

}
```

3.MULTIPLICATION:

```
#include <reg51.h>

void main(void)
{
    unsigned char d, e, f;

    d = 0x12;

    e = 0x34;

    P2 = 0x00;

    f = e * d;

    P2 = f;

}
```

4.DIVISION:

```
#include <reg51.h>

void main(void)
{
    unsigned char p, q, r;

    p = 0x12;

    q = 0x34;

    P3 = 0x00;

    r = q / p;

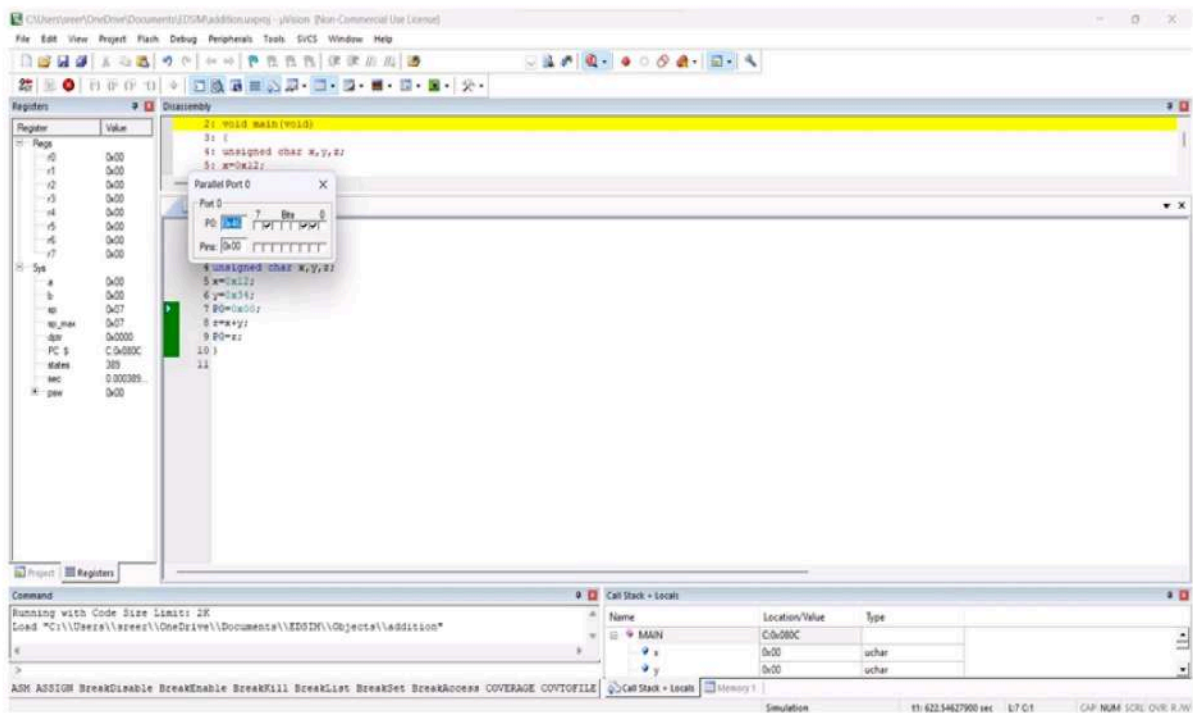
    P3 = r;

    while(1);

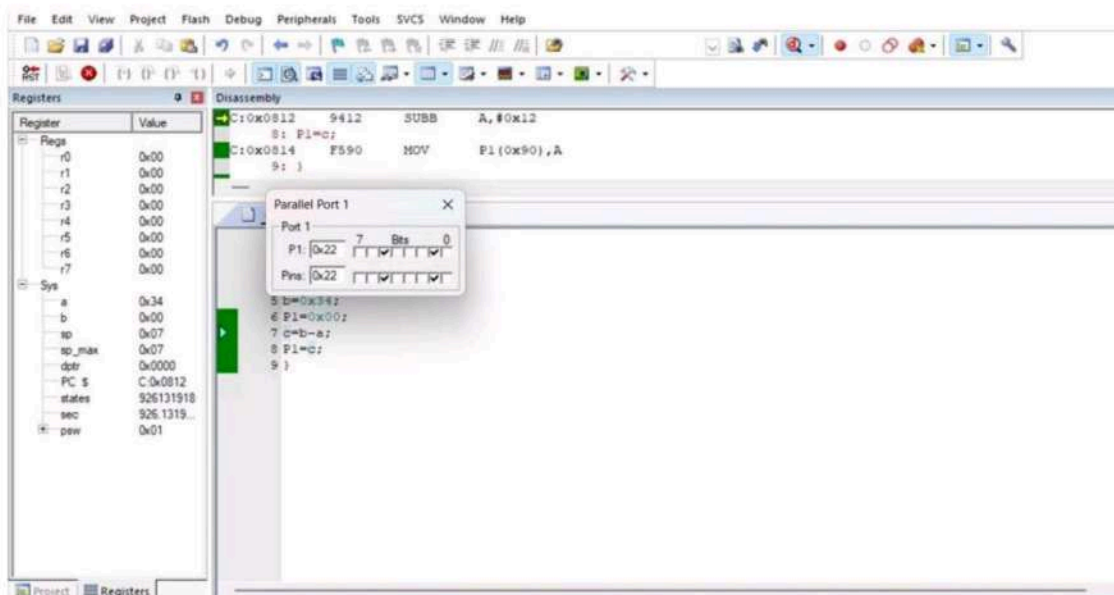
}
```


Output:

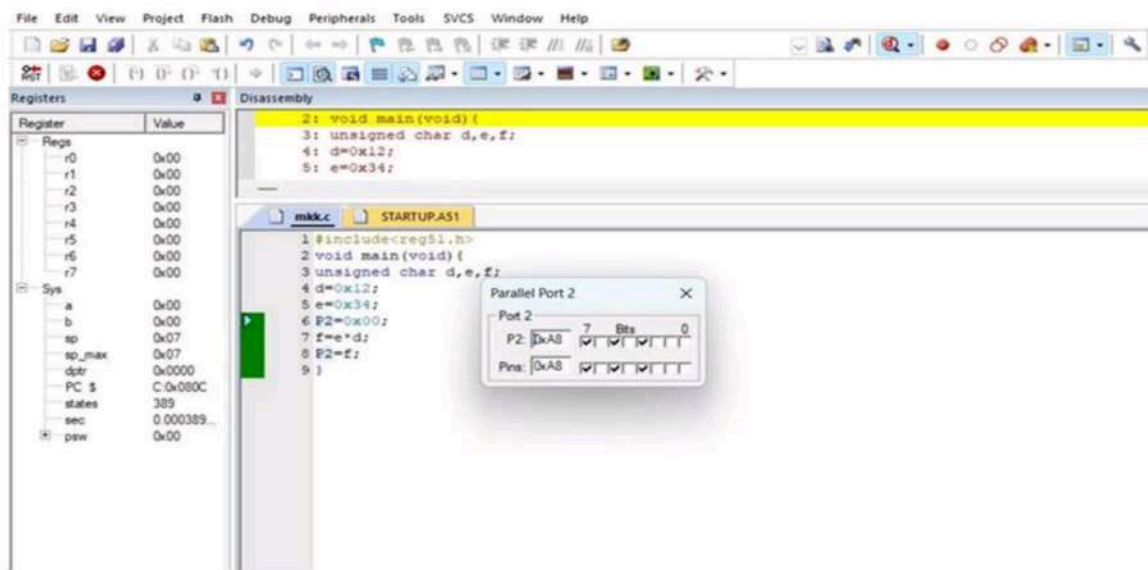
1.ADDITION:



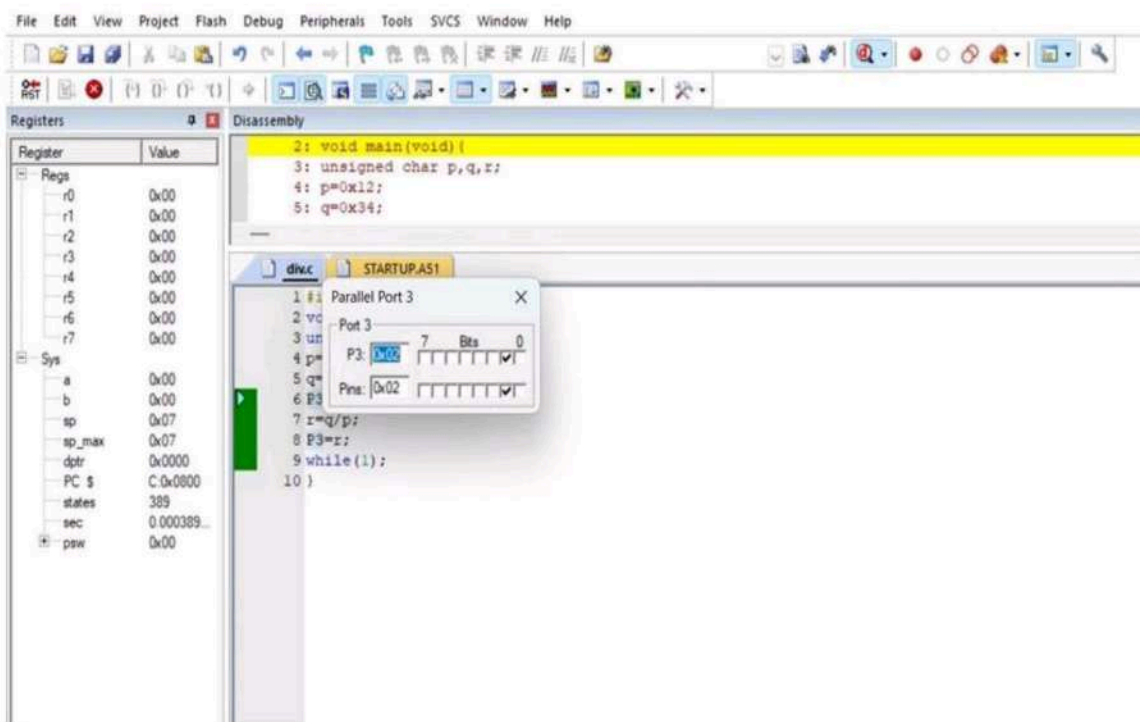
2.SUBTRACTION:



3.MULTIPLICATION:



4.DIVISION:



Result:

Thus the basic Arithmetic operations in 8051 micro-controller using Embedded- C programming was executed successfully.

EX NO :

DATE :

INTRODUCTION TO ARDUINO PLATFORM AND PROGRAMMING

Aim:

To Test Arduino Ide Using Arduino Platform And Programming.

Apparatus required:

1. Arduino board (e.g., Arduino Uno)
2. Pushbutton ,RGB LED
3. Breadboard and jumper wires
4. Downloading cable

A) PUSH BUTTON USING ARDUINO

Procedure:

1. Connect your thingZkit IoT to your computer using the USB cable.
2. Connect pin 2 of the Arduino to the onboard pushbutton (SW-2) with a pull-down configuration.
3. Ensure that the connections are secure and free from any short circuits. Setup Software, Power on the thingZkit IoT board.

Program:

```
#include<Arduino.h> // Fixed missing closing '>'

#define buttonPin 2

void setup() {

    // Initialize the button pin as an input

    pinMode(buttonPin, INPUT);

    // Initialize serial communication

    Serial.begin(9600);

    Serial.println("Active-High Push Button Status:");
```

```

}

void loop() {

  // Read the status of the button bool

  buttonState = digitalRead(buttonPin);

  // Print the button status

  if (buttonState == HIGH) {

    Serial.println("Button is not pressed");

  } else {

    Serial.println("Button is pressed (Active-High)");

  }

  delay(1000);

}

```

Diagrams:

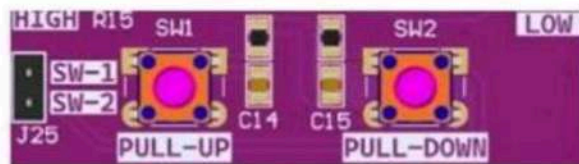
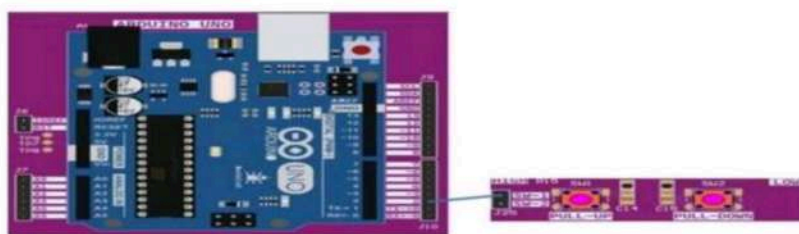


Fig no 19: General I/O Push Button

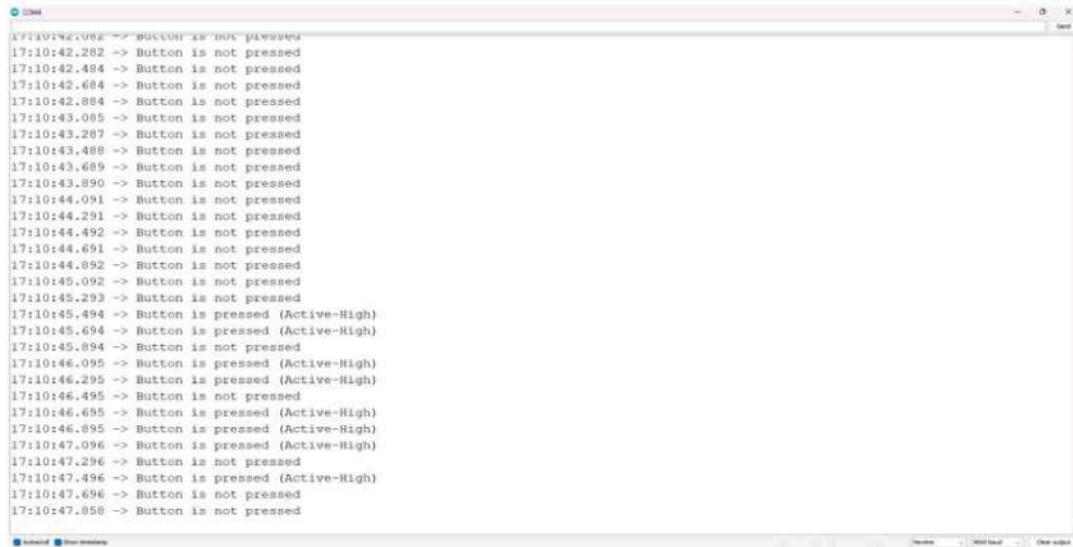
Connection Diagram:



Pin Details:

thingZkit IoT-ARDUINO	PushButton in Pull-Up Configuration
2	SW-1

Output:

A screenshot of an Arduino IDE serial monitor window. The window title is "COM4". The text inside shows a series of timestamped messages: "17:10:42.062 -> Button is not pressed", followed by several "Button is not pressed" messages with timestamps from 17:10:42.282 to 17:10:45.293. Then, a sequence of "Button is pressed (Active-High)" messages with timestamps from 17:10:45.494 to 17:10:47.496, followed by "Button is not pressed" messages with timestamps 17:10:47.696 and 17:10:47.858. The window has a "Send" button in the top right and a status bar at the bottom with "Serial" and "View history" buttons.

```
17:10:42.062 -> Button is not pressed
17:10:42.282 -> Button is not pressed
17:10:42.484 -> Button is not pressed
17:10:42.684 -> Button is not pressed
17:10:42.884 -> Button is not pressed
17:10:43.085 -> Button is not pressed
17:10:43.287 -> Button is not pressed
17:10:43.488 -> Button is not pressed
17:10:43.689 -> Button is not pressed
17:10:43.890 -> Button is not pressed
17:10:44.091 -> Button is not pressed
17:10:44.291 -> Button is not pressed
17:10:44.492 -> Button is not pressed
17:10:44.691 -> Button is not pressed
17:10:44.892 -> Button is not pressed
17:10:45.092 -> Button is not pressed
17:10:45.293 -> Button is not pressed
17:10:45.494 -> Button is pressed (Active-High)
17:10:45.694 -> Button is pressed (Active-High)
17:10:45.894 -> Button is not pressed
17:10:46.095 -> Button is pressed (Active-High)
17:10:46.295 -> Button is pressed (Active-High)
17:10:46.495 -> Button is not pressed
17:10:46.695 -> Button is pressed (Active-High)
17:10:46.895 -> Button is pressed (Active-High)
17:10:47.096 -> Button is pressed (Active-High)
17:10:47.296 -> Button is not pressed
17:10:47.496 -> Button is pressed (Active-High)
17:10:47.696 -> Button is not pressed
17:10:47.858 -> Button is not pressed
```

B) RGB LED USING ARDUINO

Procedure:

1. Connect the RGB LED pins to Arduino digital pins 9, 10, and 11 with proper resistors.
2. Initialize the pins as OUTPUT using pinMode() in the setup function.
3. In the loop, turn ON one color at a time using digitalWrite() and turn OFF others.
4. Add delay between each color change to display red, green, blue, and white sequentially.

Program:

```
#include <Arduino.h>

/* RGB LED pins (common cathode) */

#define RED_PIN 9

#define GREEN_PIN 10

#define BLUE_PIN 11

/* delay (in milliseconds) */

#define COLOR_CHANGE_DELAY 1000

void setup() {

    pinMode(RED_PIN, OUTPUT);

    pinMode(GREEN_PIN, OUTPUT);
```

```
pinMode(BLUE_PIN, OUTPUT);

}

void loop() {

    digitalWrite(RED_PIN, HIGH);

    digitalWrite(GREEN_PIN, LOW);

    digitalWrite(BLUE_PIN, LOW);

    delay(COLOR_CHANGE_DELAY);


    digitalWrite(RED_PIN, LOW);

    digitalWrite(GREEN_PIN, HIGH);

    digitalWrite(BLUE_PIN, LOW);

    delay(COLOR_CHANGE_DELAY);


    digitalWrite(RED_PIN, LOW);

    digitalWrite(GREEN_PIN, LOW);

    digitalWrite(BLUE_PIN, HIGH);

    delay(COLOR_CHANGE_DELAY);


    digitalWrite(RED_PIN, HIGH);

    digitalWrite(GREEN_PIN, HIGH);

    digitalWrite(BLUE_PIN, HIGH);

    delay(COLOR_CHANGE_DELAY);

    digitalWrite(RED_PIN, LOW);

    digitalWrite(GREEN_PIN, LOW);

    digitalWrite(BLUE_PIN, LOW);

    delay(COLOR_CHANGE_DELAY);

}
```


Diagram:

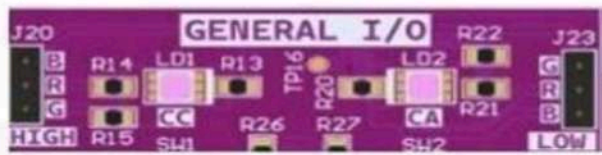
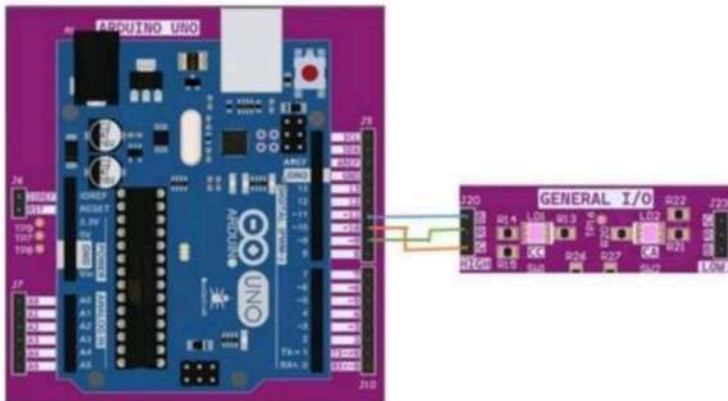


Fig no 18: General I/O RGB LED

Connection diagram:



Pin Details:

thingZkit IoT-ARDUINO	RGB LED- Common Cathode
9	Red
10	Green
11	Blue

Result:

Thus the Interfacing of button and LED with ARDUINO was performed successfully.

EX NO :

DATE :

EXPLORE DIFFERENT COMMUNICATION METHOD WITH IOT DEVICES (BLUETOOTH)

Aim :

To write a python program to explore different communication method Bluetooth with IoT devices

Apparatus required :

arduino, arduino IDE, Bluetooth.

Procedure:

1. Connect your Arduino to your computer using the USB cable. Connect the Inbuilt Bluetooth Module with Arduino as described above. Place the LED on the breadboard and connect it to the Arduino as described above.
2. Ensure that all connections are secure and free from short circuits. Setup Software: Power on the thingZkit IoT board, Open the Experiment 15 program in Arduino platform from the given source code folder.
3. Install the mobile App from the Google Play Store

Program :

```
#include <SoftwareSerial.h>

#define LED_pin 13

SoftwareSerial bluetooth(2, 3); // RX (Arduino pin 2), TX (Arduino pin 3)

float data = 25.98;

char charArray[10];

void setup() {

  Serial.begin(9600);    // Initialize the Serial Monitor
```

```

bluetooth.begin(9600);    // Initialize Bluetooth communication

pinMode(LED_pin, OUTPUT); // Set LED pin as OUTPUT

}

void loop()

{ // Check if data is available from the Bluetooth module

if (bluetooth.available()) {

char receivedChar = bluetooth.read(); // Read the character received via Bluetooth

Serial.print("Received: ");

Serial.println(receivedChar);

if (receivedChar == '1')

{

digitalWrite(LED_pin, HIGH);

Serial.println("LED_ON");

}

else if (receivedChar == '0')

{

digitalWrite(LED_pin, LOW);

Serial.println("LED_OFF");

}

else if (receivedChar == '2')

{

dtostrf(data, 6, 2, charArray); // Convert float to char array

bluetooth.write(charArray);    // Send data back via Bluetooth

delay(100);

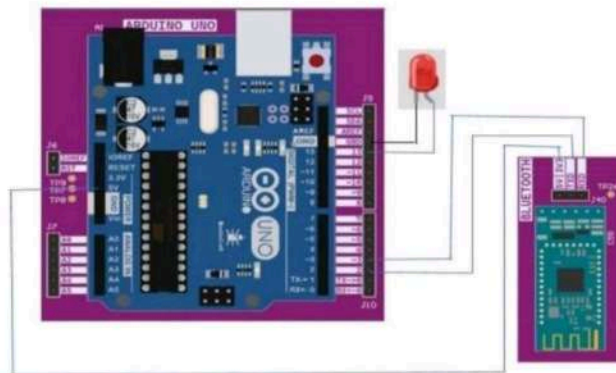
}

}

}

```

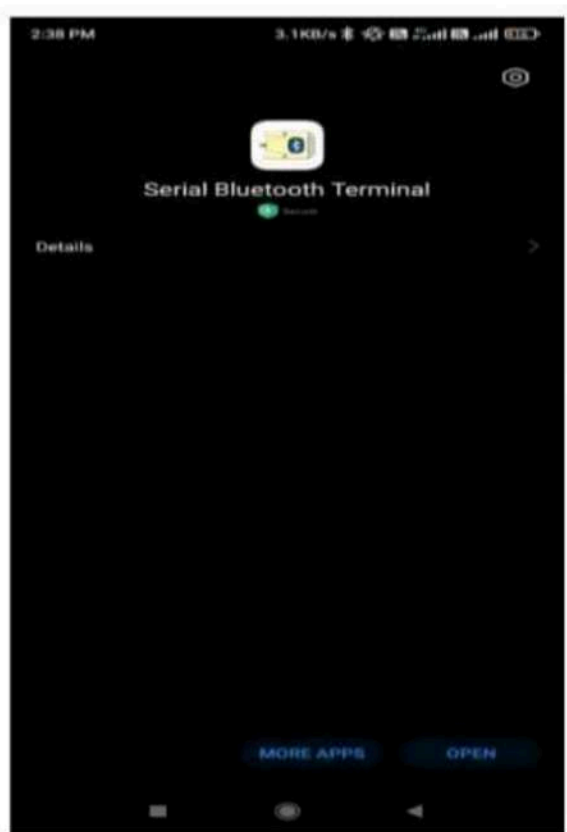
Connection:



Pin Details:

thingZkit IoT-ARDUINO	Bluetooth Module & LED
Pin 2	TX
Pin 3	RX
5V	5V/3V
Pin 13	External LED (Long leg terminal)
GND	LED Short leg terminal

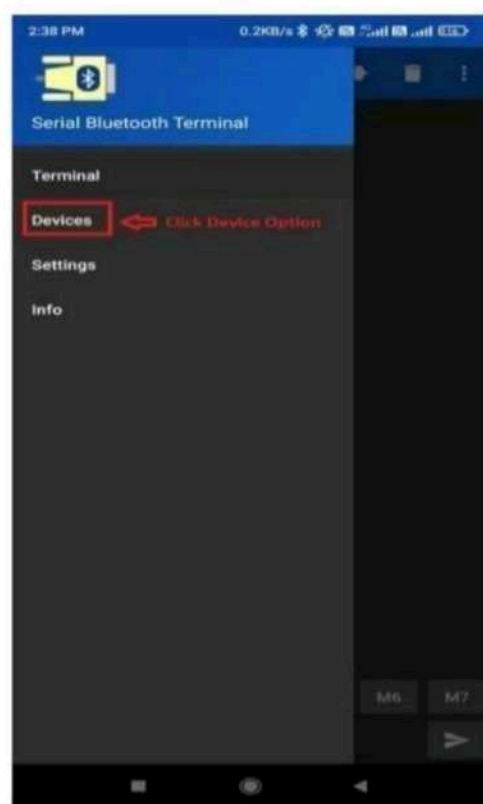
Output:

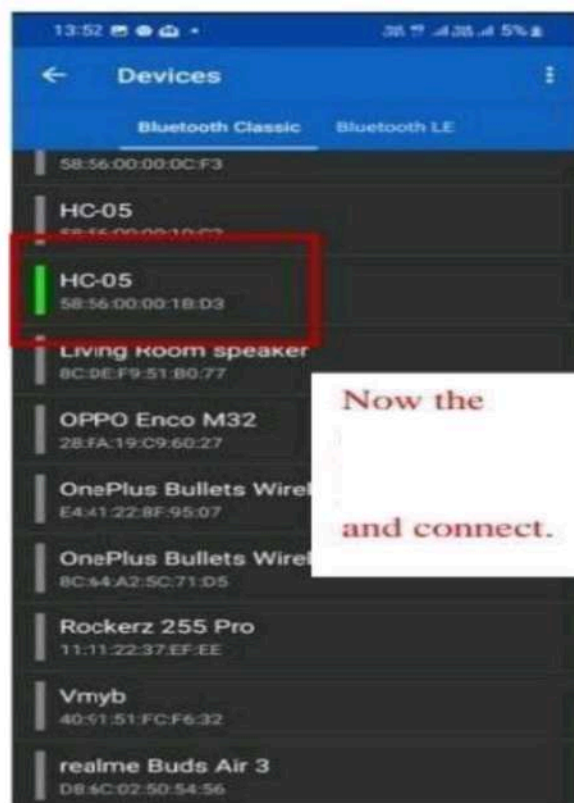
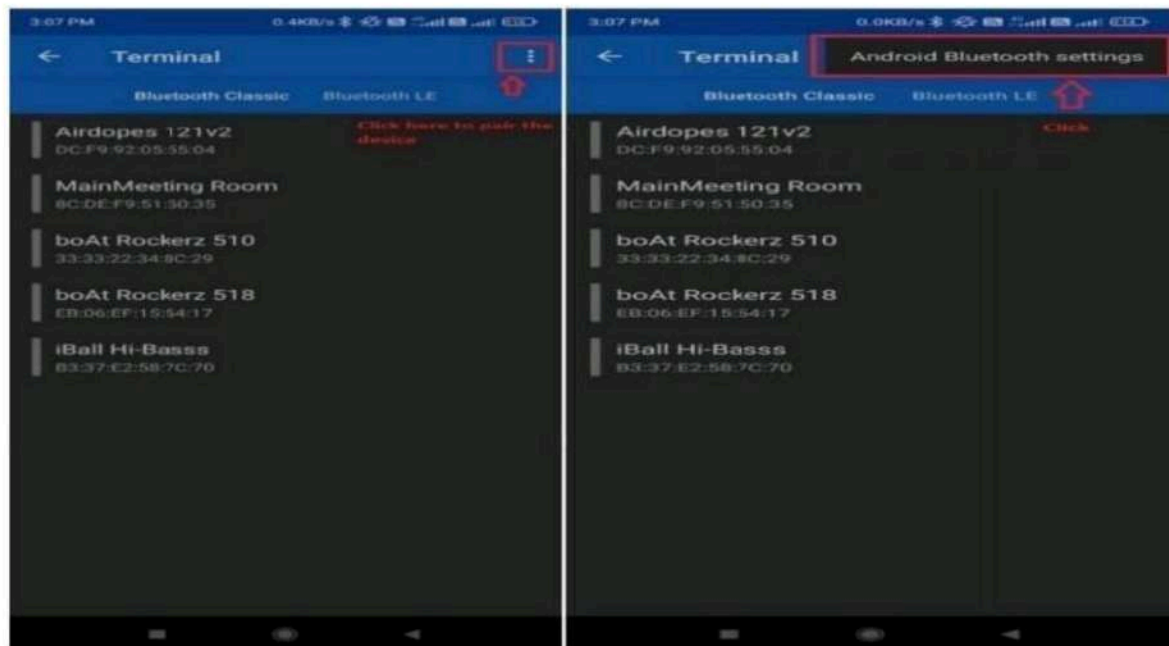


1. Open the Arduino Serial Monitor from the Arduino IDE.
2. Ensure that the baud rate in the Serial Monitor matches the baud rate in the Arduino code (in this case, 9600).

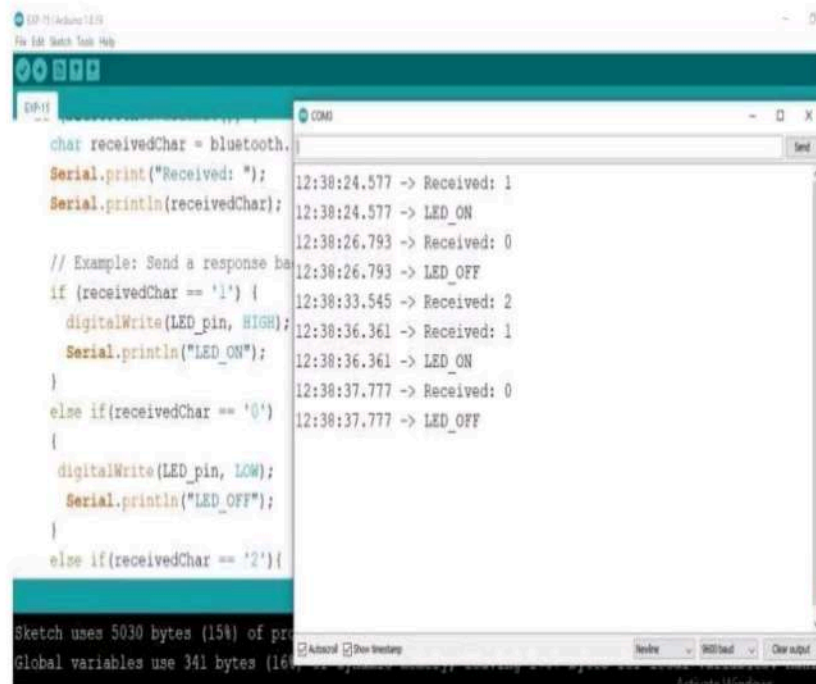
Turn on Bluetooth in mobile. Open the App and follow the steps as mentioned in the below

Screenshots:









The screenshot shows the Arduino IDE interface. The main window displays a C++ sketch for controlling an LED via Bluetooth. The sketch includes a variable `receivedChar` of type `char` initialized to `bluetooth`. It uses `Serial.print` and `Serial.println` to output received data. The logic checks for received characters '1', '0', and '2' to turn the LED on or off using `digitalWrite`. The status bar at the bottom indicates the sketch uses 5030 bytes (15%) of program space and 341 bytes (16%) of global variables space.

```
char receivedChar = bluetooth;
Serial.print("Received: ");
Serial.println(receivedChar);

// Example: Send a response back
if (receivedChar == '1') {
  digitalWrite(LED_pin, HIGH);
  Serial.println("LED_ON");
}
else if (receivedChar == '0') {
  digitalWrite(LED_pin, LOW);
  Serial.println("LED_OFF");
}
else if (receivedChar == '2') {
```

Serial Monitor Output:

```
12:38:24.577 -> Received: 1
12:38:24.577 -> LED_ON
12:38:26.793 -> Received: 0
12:38:26.793 -> LED_OFF
12:38:33.545 -> Received: 2
12:38:36.361 -> Received: 1
12:38:36.361 -> LED_ON
12:38:37.777 -> Received: 0
12:38:37.777 -> LED_OFF
```

Sketch uses 5030 bytes (15%) of program space (maximum 32768 bytes).
Global variables use 341 bytes (16%) of dynamic memory (maximum 2048 bytes).

Result:

Thus the python program to explore different communication method with IoT devices (Bluetooth) was written and executed successfully

EX NO :

DATE :

INTRODUCTION TO RASPBERRY PI PLATFORM AND PYTHON PROGRAMMING

Aim:

Connect a LED with Raspberry Pi with thonny python ide of the Raspberry Pi.

Appartus required :

- Raspberry pi board
- Blinking LED
- Power supply
- Cables
- Internet Connectivity

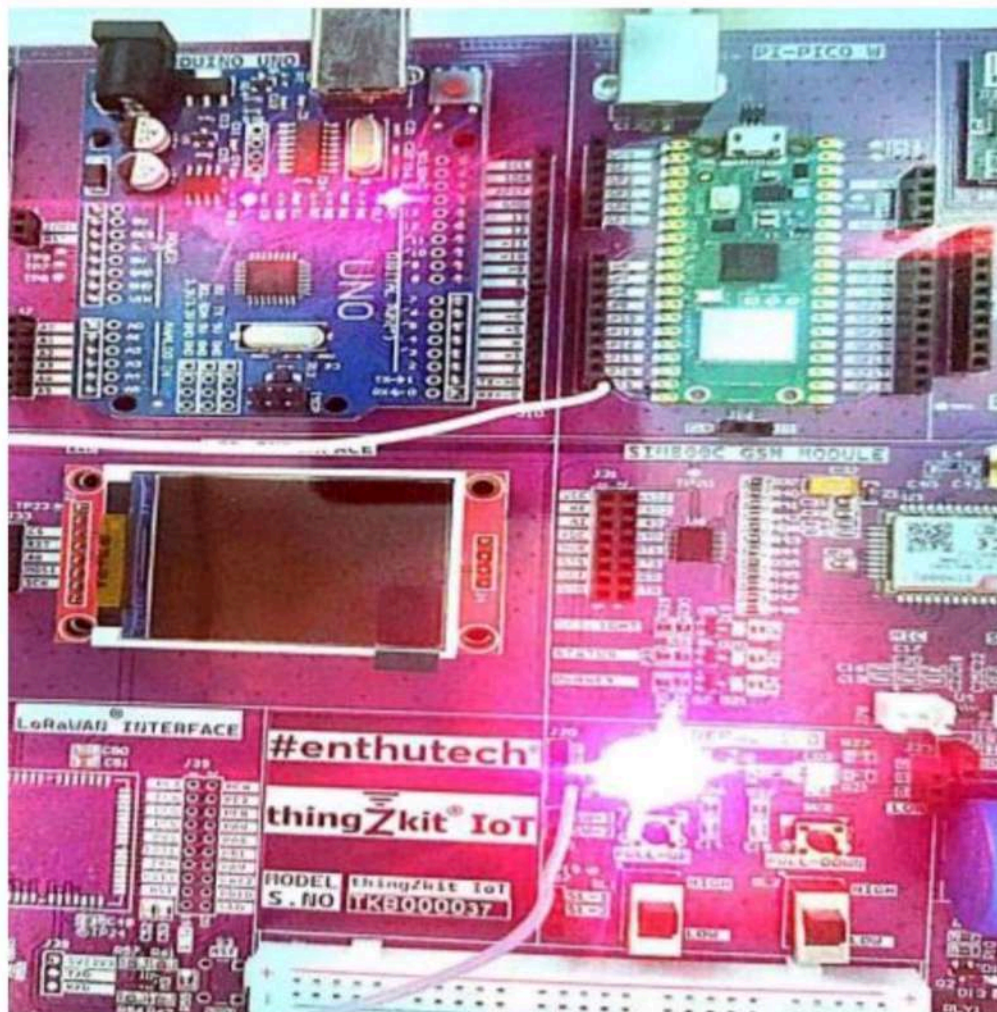
Procedure:

1. Using thonny python ide and create code in python format.
2. Connect GPIO 17 to LED.
3. Run the python ide

Program:

```
from machine import Pin
import time
led = Pin(17, Pin.OUT) # GPIO 17 on Pico
while True:
    led.value(1)
    print("LED ON")
    time.sleep(1)
    led.value(0)
    print("LED OFF")
    time.sleep(1)
```

Output:



Result:

Thus the Interfacing of LED with Raspberry Pi platform was performed successfully.

EX NO :

DATE :

INTERFACING SENSOR WITH RASPBERRY PI

Aim:

Interfacing the ultrasonic sensor with a Raspberry Pi involves connecting the sensor to the GPIO pins of the Raspberry Pi.

Apparatus required:

- Raspberry pi board
- ultrasonic sensor
- Blinking LED
- power supply
- cables
- Internet Connectivity

Procedure:

- Connect the VCC pin of the ultrasonic sensor to the 5V pin on the Raspberry Pi.
- Connect the TRIG pin of the ultrasonic sensor to a GPIO 14 pin on the Raspberry Pi.
- Connect the ECHO pin of the ultrasonic sensor to the GPIO 15 pin on the Raspberry Pi.
- Save the Python script with a .py extension .
- Run the python ide. Verify the distance.

Program:

```
from machine import Pin, time_pulse_us

import time

# Define pins
```

```
TRIG = Pin(14, Pin.OUT) # example: GPIO14

ECHO = Pin(15, Pin.IN) # example: GPIO15

def get_distance():

    # Make sure trigger is low

    TRIG.value(0)

    time.sleep_us(2)

    # Trigger high for 10us

    TRIG.value(1)

    time.sleep_us(10)

    TRIG.value(0)

    # Measure the echo pulse

    duration = time_pulse_us(ECHO, 1, 30000)

    if duration < 0:

        return None

    distance_cm = (duration / 2) * 0.0343

    return distance_cm

while True:

    distance = get_distance()

    if distance is not None:

        print("Distance: {:.2f} cm".format(distance))

    else:

        print("Out of range")

    time.sleep(1)
```


Output:

```
1 from machine import Pin, time_pulse_us
2 import time
3
4 # Define pins
5 TRIG = Pin(14, Pin.OUT) # example: GPIO3
6 ECHO = Pin(15, Pin.IN)  # example: GPIO2
7
8 def get_distance():
9     # Make sure trigger is low
10    TRIG.value(0)
11    time.sleep_us(2)
12
13    # Trigger high for 10us
14    TRIG.value(1)
15    time.sleep_us(10)
16    TRIG.value(0)
17
18    # Measure the echo pulse
19    duration = time_pulse_us(ECHO, 1, 30000) # timeout 30ms
20    if duration < 0:
21        return None # Timeout occurred
22
23    # Calculate distance (speed of sound: 343 m/s)
24    distance_cm = (duration / 2) * 0.0343
25    return distance_cm
26
27 # Main loop
28 while True:
29     distance = get_distance()
30     if distance is not None:
31         print("Distance: {:.2f} cm".format(distance))
32     else:
33         print("Out of range")
34     time.sleep(1)
35
```

```
Shell x
Distance: 244.27 cm
Distance: 243.58 cm
Distance: 244.04 cm
Distance: 244.66 cm
Distance: 244.18 cm
Distance: 244.34 cm
```

Result:

Thus the python program to interface ultrasonic sensors with Raspberry PI was written and executed successfully.

EX NO :

DATE :

STUDY OF ARDUINO TO PI WIRELESS COMMUNICATION

Aim :

To study wireless communication between arduino and Pi pico W .

Apparatus required :

1. Arduino UNO.
2. PICO W .
3. Jumpers
4. TFT Display.

Procedure :

1. UPLOAD TX Code to PI PICO W using Thonny python IDE .
2. Upload Rx Code to Arduino UNO using Arduino IDE.
3. Change the wifi SSID and PASSWORD in both code .
4. Ones connected enter the IP address for server in both codes.
5. Connect as per the connection diagram.
6. Run the code .

Program :

Tx Code In PICO W:

```
import network  
  
import socket  
  
import time
```

```
# Wi-Fi credentials

ssid = 'WIFINAME'

password = 'PASSWORD'

# Server details (Arduino with ESP8266/ESP32)

server_ip = '192.168.238.10' # Replace with your server IP

server_port = 80          # Must match Arduino server port

# Connect to Wi-Fi

wlan = network.WLAN(network.STA_IF)

wlan.active(True)

wlan.connect(ssid, password)

# Wait until connected

while not wlan.isconnected():

    print("Connecting to Wi-Fi...")

    time.sleep(1)

print("Connected to Wi-Fi")

print(wlan.ifconfig()) # Print IP configuration

# Create TCP socket

client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Connect to server

try:

    client_socket.connect((server_ip, server_port))

    print("Connected to the server.")

except Exception as e:

    print("Failed to connect to the server:", e)

    client_socket.close()
```

```

# Message to send

send_message = "tq team"

# Send data continuously

while True:

    try:

        # Send message

        client_socket.send(send_message.encode('utf-8'))

        print("Sent:", send_message)


        # Receive acknowledgment

        ack = client_socket.recv(1024)

        print("Received:", ack.decode('utf-8'))

        time.sleep(3)

    except Exception as e:

        print("Error:", e)

        client_socket.close()

        break

# Close socket

client_socket.close()

```

Rx Code for Arduino UNO:

```

#include <SoftwareSerial.h>

#define RX 2

#define TX 3

SoftwareSerial esp8266(RX, TX);

// Wi-Fi credentials

String AP = "WIFINAME";

```

```
String PASS = "PASSWORD";
```

```
// Server configuration
```

```
String PORT = "80";
```

```
int countTrueCommand;
```

```
int countTimeCommand;
```

```
boolean found = false;
```

```
void setup() {
```

```
    Serial.begin(9600);
```

```
    esp8266.begin(115200);
```

```
    // Initialize ESP8266
```

```
    sendCommand("AT", 5, "OK");
```

```
    // Set mode: SoftAP + Station
```

```
    sendCommand("AT+CWMODE=3", 5, "OK");
```

```
    // Connect to Wi-Fi
```

```
    sendCommand("AT+CWJAP=\"" + AP + "\",\"" + PASS + "\"", 80, "OK");
```

```
    // Enable multiple connections
```

```
    sendCommand("AT+CIPMUX=1", 5, "OK");
```

```
    // Start server
```

```
    sendCommand("AT+CIPSERVER=1," + PORT, 5, "OK");
```

```
    Serial.println("Server started. Waiting for clients...");
```

```
}
```

```
void loop() {
```

```
    if (esp8266.available()) {
```

```
String response = esp8266.readString();
```

```
    Serial.println("Received data: " + response);
```

```
    // Check for incoming data
```

```
    if (response.indexOf("+IPD") >= 0) {
```

```
int start = response.indexOf(":") + 1;

String data = response.substring(start);

Serial.println("Message: " + data);

}

}

}
```

// Function to send AT commands

```
void sendCommand(String command, int maxTime, char readReplay[]) {

    Serial.print(countTrueCommand);

    Serial.print(". AT command => ");

    Serial.print(command);

    Serial.print(" ");

    while (countTimeCommand < (maxTime * 1)) {

        esp8266.println(command);

        if (esp8266.find(readReplay)) {

            found = true;

            break;

        }

        countTimeCommand++;

    }

    if (found) {

        Serial.println("OK");

        countTrueCommand++;

        countTimeCommand = 0;

    } else {

        Serial.println("Failed");

    }

}
```

```
countTrueCommand = 0;
```

```
countTimeCommand = 0;
```

```
}
```

```
found = false;
```

```
}
```

Result :

Thus Tested was Successfully Completed.

EX NO :

DATE :

STUDY OF SETUP A CLOUD PLATFORM TO LOG THE DATA

Aim:

To study to setup a cloud platform to log the data.

Apparatus Required:

1. ARDUINO IDE
2. NODE MCU
3. IR SENSOR
4. Bread board

Procedure:

- Step 1: Connect the IR sensor to the microcontroller.
- Step 2: Create a channel in ThingSpeak and get the API key.
- Step 3: Install Arduino IDE and required libraries.
- Step 4: Write code to read IR sensor data and connect to Wi-Fi.
- Step 5: Send data to ThingSpeak and upload the code.
- Step 6: View the data on the ThingSpeak graph

Screenshots:

Communicating Between D... How to connect Raspberry (T) WhatsApp ChatGPT Sign in - ThingSpeak IoT

thingspeak.com/login?skipSSOCheck=true

ThingSpeak™ Channels Apps Support + Commercial Use How to Buy

non-commercial users may use ThingSpeak for free. Free accounts offer limited access to certain functionality. Commercial users are eligible for a time-limited free evaluation. To get full access to the MATLAB analysis features on ThingSpeak, log in to ThingSpeak using the email address associated with your university or organization.

To send data faster to ThingSpeak or to send more data from more devices, consider the [paid license options](#) for commercial, academic, home and student usage.

Create MathWorks Account

Email Address

Location United States

First Name

Last Name

Continue

DATA AGGREGATION AND ANALYTICS ThingSpeak

SMART CONNECTED DEVICES

MATLAB

ALGORITHM DEVELOPMENT SENSOR ANALYTICS

This website uses cookies to improve your user experience, personalize content and ads, and analyze website usage. By continuing to use this website, you consent to our use of cookies. Please see our [Privacy Policy](#) to learn more about cookies and how to change your settings.

Activate Windows Go to Settings to activate Windows.

DOC-20250513-WA0027.pdf WhatsApp ChatGPT sign in thingspeak - Search Sign Up Success - ThingSpeak IoT

https://thingspeak.mathworks.com/static_pages/signup_complete_success

ThingSpeak™ Channels Apps Devices Support + Commercial Use How to Buy

Sign-up successful

Congratulations, you have successfully linked your MathWorks account to ThingSpeak. Use the following email ID and its associated MathWorks account password on all subsequent logins to ThingSpeak.

Email ID: suashi78@gmail.com

Welcome to ThingSpeak!

OK

Program :

```
#include <IRremote.h>

#include <ESP8266WiFi.h>

#include <ThingSpeak.h>

const char *ssid = "silicon systems";

const char *password = "Silicon@2017";

const char *apiKey = "QWD5TTQ08RBGQFVH";
```

```
const int IR_PIN = 2; // Change this to the pin connected to your IR sensor

IRrecv irrecv(IR_PIN); decode_results results;

void setup() {  Serial.begin(9600);

  irrecv.enableIRIn(); // Start the IR receiver

  WiFi.begin(ssid,password);

  ThingSpeak.begin(client); // Initialize ThingSpeak

}

void loop() {

  if (irrecv.decode(&results)) {

    // Read and send the data to ThingSpeak

    int irValue = results.value;

    Serial.print("IR Value: "); Serial.println(irValue);

    // Send data to ThingSpeak

    ThingSpeak.writeField(YOUR_CHANNEL_ID, 1, irValue, apiKey);

    irrecv.resume(); // Receive the next value

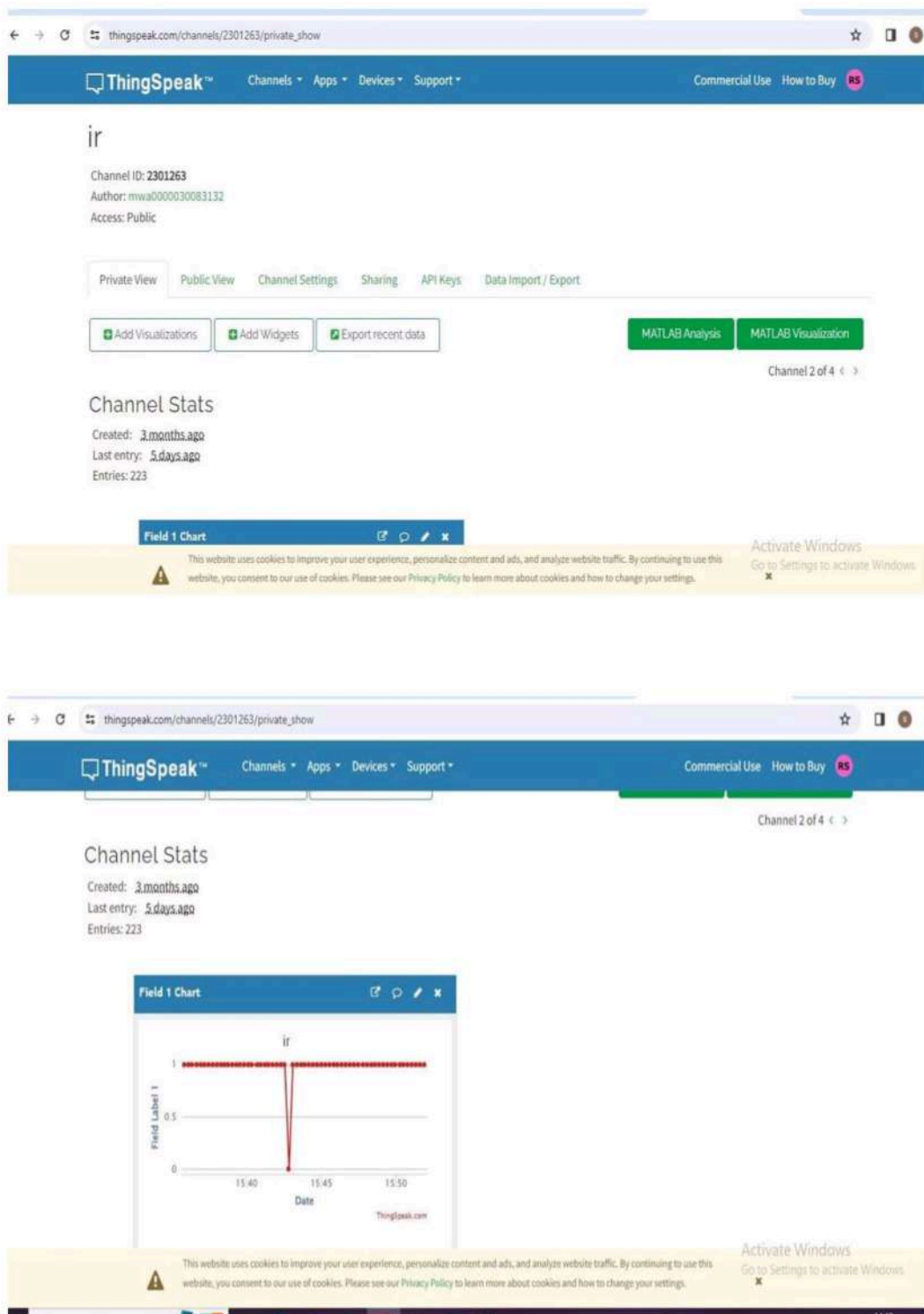
    delay(10000); // Delay to avoid sending data too frequently

  }

  // Other loop logic can be added here

}
```

Output:



Result :

The IR sensor data was successfully sent to ThingSpeak and displayed in graphical form for real-time monitoring.

EX NO :

DATE :

STUDY OF LOG DATA USING RASPBERRY PI AND UPLOAD TO THE CLOUD PLATFORM

Aim :

To study test the log data using raspberry pi and upload to the cloud platform.

Apparatus required:

1. RASPBERRY PI
2. NODE MCU
3. IR SENSOR
4. Bread board

Procedure :

1. Connect the IR sensor to the Arduino.
2. The connections may vary based on your specific IR sensor model, but typically, it involves connecting the sensor's signal pin to a digital pin on the Arduino and power/ground pins appropriately.
3. Install the necessary libraries for the IR sensor.
4. For example, if you are using a common IR sensor like the TSOP382, you might need the "IRremote" library.

Program :

```
import machine

import utime

import urequests

import network

IR_SENSOR_PIN = 2 # GPIO pin for the IR sensor

API_KEY = "ZSX5SRTSRJ61XVU6"

THINGSPEAK_URL =

"https://api.thingspeak.com/update?api_key={}".format(API_KEY)

WIFI_SSID = "@ms.Balaji"

WIFI_PASSWORD = "@msbalaji"

ir_sensor = machine.Pin(IR_SENSOR_PIN, machine.Pin.IN)

wlan = network.WLAN(network.STA_IF)

def connect_to_wifi(): wlan.active(True)

if not wlan.isconnected(): print("Connecting to WiFi...")

wlan.connect(WIFI_SSID, WIFI_PASSWORD) while not wlan.isconnected():

pass

print("Connected to WiFi")

def read_ir_sensor():

    return ir_sensor.value()

def send_to_thingspeak(data):

# Send data to ThingSpeak

url = "{}&field1={}".format(THINGSPEAK_URL, data)

response = urequests.get(url)

print("ThingSpeak response:", response.text)

# Connect to WiFi connect_to_wifi()
```

while True:

```
    ir_data = read_ir_sensor()
```

```
    print("IR Sensor Data:", ir_data)
```

```
# Send data to ThingSpeak via HTTP
```

```
send_to_thingspeak(ir_data)
```

```
    utime.sleep(15) # Wait for 15 seconds before the next reading
```

Output :

Communicating Between D... X How to connect Raspberry X (1) WhatsApp X ChatGPT X Sign In - ThingSpeak IoT X +

thingspeak.com/login?skipSSOcheck=true

ThingSpeak™ Channels Apps Support + Commercial Use How to Buy

Non-commercial users may use ThingSpeak for free. Free accounts have limits on certain functionality. Commercial users are eligible for a time-limited free evaluation. To get full access to the MATLAB analysis features on ThingSpeak, log in to ThingSpeak using the email address associated with your university or organization.

To send data faster to ThingSpeak or to send more data from more devices, consider the [paid license options](#) for commercial, academic, home and student usage.

Create MathWorks Account

Email Address

Location

United States

First Name

Last Name

Continue

DATA AGGREGATION AND ANALYTICS ThingSpeak

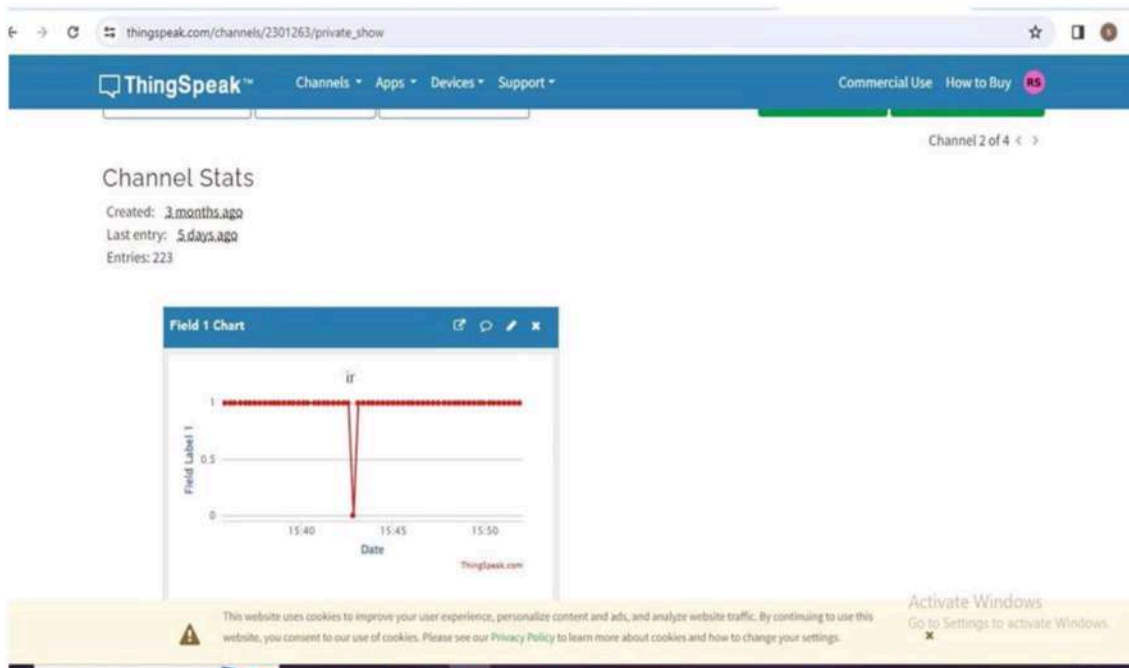
SMART CONNECTED DEVICES

MATLAB

ALGORITHM DEVELOPMENT SENSOR ANALYTICS

This website uses cookies to improve your user experience, personalize content and ads, and analyze website traffic. By continuing to use this website, you consent to our use of cookies. Please see our [Privacy Policy](#) to learn more about cookies and how to change your settings.

Activate Windows
Go to Settings to activate Windows.



thingspeak.com/channels

ThingSpeak™ Channels Apps Devices Support Commercial Use How to Buy RS

Signed in successfully.

My Channels

New Channel

Search by tag

Name	Created	Updated
demo_wdm Private Public Settings Share API Key Data Import / Export	2023-07-08	2023-07-10 06:24
led_control Private Public Settings Share API Key Data Import / Export	2023-07-11	2023-07-11 05:27
lr Private Public Settings Share API Key Data Import / Export	2023-10-12	2023-10-30 08:00

Help

Collect data in a ThingSpeak channel from a device, from another channel, or from the web.

Click **New Channel** to create a new ThingSpeak channel.

Click on the column headers of the table to sort by the entries in that column or click on a tag to show channels with that tag.

Learn to create channels, explore and transform data.

Learn more about [ThingSpeak Channels](#).

Examples

- [Arduino](#)

This website uses cookies to improve your user experience, personalize content and ads, and analyze website traffic. By continuing to use this website, you consent to our use of cookies. Please see our [Privacy Policy](#) to learn more about cookies and how to change your settings.

Activate Windows
Go to Settings to activate Windows.

Result :

Thus the study of log data using Raspberry Pi and upload to the cloud platform was Successfully tested and Completed.